A Release Test Is Not Runtime Authority: Evidence Reuse at Runtime-Assurance Authority Transitions

Abstract

A learning-enabled controller can adapt while runtime assurance retains action authority, then release a raw policy after a finite check. Such evidence is tied to the checked subject, authority, and horizon. In Safe-Control-Gym, a clean cohort of 20 independently trained controllers supplied 480 accepted snapshot–state pairs. Permanent raw release failed on 10 pairs across five training runs, whereas resident predictive authority failed on none. A bounded reward-record attacker widened this gap while satisfying calibrated known-sign and scalar-envelope checks. After 48 protected update batches, it caused 46/116 failures under raw release and 34/116 under resident authority. Paired accounting found 27 failures under both contracts, 19 only under raw release, seven only under resident authority, and 63 under neither. Every resident failure followed a switch to the trusted baseline, with a median of one further step to violation. Thus, repeated raw-policy checks did not also establish the safety of the baseline handoff. Bootstrap-only poisoning also increased retirement violations in two of three provenance-complete pairs of a published lifecycle. Trusted recomputation closes the reward channel, but each controller receiving authority still requires evidence for its own deployment contract.

1 Introduction

Runtime assurance is a standard way to let a controller learn without letting it crash the plant. A Simplex decision module, a runtime shield, or a model-predictive safety filter holds action authority during adaptation: it replaces unsafe policy proposals with actions from a trusted baseline. In our reported adaptation runs, every executed transition satisfied the measured safety constraints [1–3]. Deployments need not keep that machinery forever. Published learning-enabled control workflows study retirement of the protective authority once commissioning succeeds, promoting the raw policy snapshot to a lower-latency control target and removing permanent computation, intervention, and dependence on a backup service [4, 5]. A finite release check (a bounded-horizon prediction, a certified rollout, a commissioning test) authorizes that promotion.

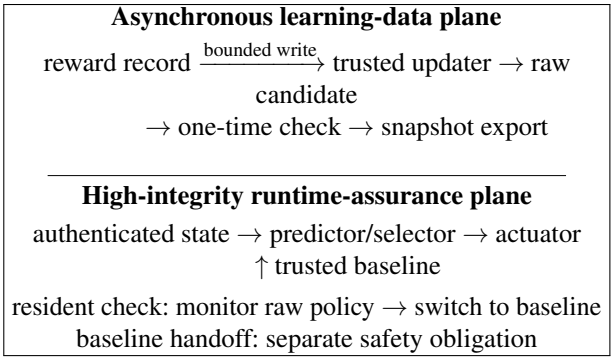

Figure 1: Evaluated trust boundary. The benchmark begins after bounded reward-record access is obtained; it does not model the initial service compromise. Upstream provenance or recomputation removes this attack path, whereas resident predictive authority also requires evidence that the trusted baseline can safely accept the state at handoff.

We show that a finite release check does not transfer across the retirement it authorizes. Protection establishes properties of the executed composition; raw release changes both the controller with action authority and the horizon over which safety is required. Repeating the check online defines a different contract. It observes the raw policy over a receding horizon, but a switch to the trusted baseline introduces another controller whose response from the handoff state requires separate evidence. We call the general flaw *evidence reuse across an authority transition*: evidence produced under one deployment contract is reused after the certified subject, authority, provenance, information, or horizon has changed. The controlled experiments study two such transitions: release from the assured composition to the raw policy, and transfer from resident predictive authority to the trusted baseline. Reward-record corruption additionally changes provenance.

A clean cohort fixes the ordering without an attacker.

Twenty independently trained controllers supplied 480 snapshot–state pairs accepted by the clean five-step check. Permanent raw release failed on 10 pairs distributed across five training runs, whereas resident predictive authority failed on none. All 10 paired discordances favoured residence (two-sided exact $p = 0.00195$), and the run-level failure frequency was 5/20 (Wilson 95% CI $[11.2\%, 46.9\%]$). State pairs within a trained controller are repeated measurements, so the run count, not 480, supplies training-level replication. No attacker is involved in this separation.

An upstream attacker widens the gap rather than creating it. While the shield masks unsafe proposals, corrupted update data steers the learner without a physical violation during the reported adaptation runs. Bounded reward-record edits raised a separate 120-pair count from 2 to 27. A sign/envelope-constrained attacker remained effective as the protected bootstrap grew. At 48 update batches it caused 46/116 raw-release failures and 34/116 failures under resident predictive authority. Paired outcomes were 27 failures under both contracts, 19 only under raw release, seven only under resident authority, and 63 under neither. All 34 resident failures followed a switch to the trusted baseline, and violation occurred after a median of one additional step. The seven resident-only failures show that detecting risk in the raw policy did not establish the safety of the controller activated at the handoff. We call the attack *shield-retirement poisoning*.

The attack applies when the runtime plane cannot reconstruct reward semantics from authenticated transitions (Section 3). Trusted recomputation removes the channel when such reconstruction is available. It does not remove the attacker-independent release gap or establish the safety of a subsequent baseline handoff.

Operational technology (OT) guidance separates runtime-control assets from the data-management services and historians that exchange information across zones [6, 7], so we make the attacker deliberately narrow in authority and start the threat model after compromise of a reward-ingestion principal in that learning-data plane. This is an explicit split-trust precondition: the attacker may modify bounded scalar reward records consumed by a policy-gradient updater, and reads the learner state those edits are computed against. Everything on the authenticated control path stays outside its write authority (Table 1, Fig. 1).

Simplex and its neural and barrier-based successors keep a decision module resident [1, 8, 9]. Policy repair and safe projection constrain a learned policy before deployment [10–12]. Reward-poisoning work studies how adversarial learning signals redirect updates, but not behind a shield. The literature has reached both sides of the deployment decision: shielded-learning studies find that a shielded learner need not become intrinsically safe and recommend retaining the shield during execution [13], while published workflows bootstrap behind a shield and then retire it [4, 5]. The open question is therefore not whether shielding helps, but what evidence justifies handing action authority to the raw policy, and what separately justifies a later handoff to the baseline. Those decisions are what we study.

1.1 Evidence and Contributions

The contract gap and the channel that widens it are separate objects. The clean Safe-Control-Gym cohort estimates how often the release gap appeared across independent training runs. A horizon sweep measures the warning provided by the resident raw-policy check. The duration sweep then evaluates the distinct handoff to the trusted baseline. Carr et al.'s published workflow provides an independent shield-retirement lifecycle [4]. Adding bounded reward-record poisoning only during its protected bootstrap increased raw-policy violations at retirement by factors of 3.6 and 1.6 in two of three provenance-complete paired REINFORCE training runs.

Our contributions are:

- We decompose release-time acceptance into an evidence tuple over subject, authority, provenance, information, and horizon, and use it to derive the obligations a deployment must re-establish for every component the authority transition changes.
- We estimate the attacker-independent gap over 20 independently trained controllers, separating one-time raw release from resident predictive authority on paired clean snapshots. A locked horizon sweep quantifies the warning produced by the repeated raw-policy check.
- We instantiate bounded reward-record writes in the controlled study and in a third-party shield-retirement lifecycle. In the sign/envelope-constrained duration sweep, resident authority prevented 19 raw-release failures but had seven resident-only failures. The result isolates the baseline handoff as a separate evidence obligation.
- We separate controls by contract. Trusted recomputation closes the upstream reward channel; commit admission restricts raw-policy release; resident authority requires an independently supported baseline handoff; and adaptation should be deployed only when paired task benefit justifies its transition risks and costs. Our evaluated benign shifts do not establish such a benefit.

2 Background and Problem Model

2.1 Attacked Histories and Plausible States

We consider a discrete-time cyber-physical system

$$x_{t+1} = f(x_t, u_t, w_t), \qquad \widetilde{y}_t = h(x_t) + a_t, \tag{1}$$

where x_t is the physical state, u_t is the control input, $w_t \in \mathcal{W}$ is ordinary process uncertainty, and a_t is a false-data injection

(FDI) signal on the measurement channel. The controller observes the attacked history

$$h_t = (\widetilde{y}_{0:t}, u_{0:t-1}, \widetilde{\ell}_{0:t}), \tag{2}$$

where $\widetilde{\ell}_t = \ell_t + b_t$ is a learning signal modified by update-data corruption b_t. The general model covers rewards, logs, and auxiliary learning signals; the end-to-end experiments specialize this channel to bounded scalar reward-record edits. Sensor FDI creates uncertainty about the physical state, whereas update-data corruption can redirect learning even with trusted state observations. Either channel can change the action map produced from h_t.

Following severe sensor-attack safety work [14], an attacked history may correspond to multiple physical states. For an attack family $\mathcal{A}$, define the plausible-state set

$$X_{\mathcal{A}}(h_t) = \{x : \exists E \in \mathcal{A},\, H_t(E) = h_t,\, x_t(E) = x\}. \tag{3}$$

This set defines the state semantics of a certificate under strategic FDI: a certificate evaluated only at a nominal estimate $\hat{x}(h_t)$ does not cover another compatible state in $X_{\mathcal{A}}(h_t)$.

2.2 Online-Updated Controllers and Certified Kernels

The controller is a history-dependent policy π_{θ_t} that selects $u_t = \pi_{\theta_t}(h_t)$ and updates according to $\theta_{t+1} = U(\theta_t, h_t)$. An update is *nontrivial* when it changes the controller's action map on a reachable or admissible history; parameter changes that preserve the action map are behaviorally trivial.

Let Γ denote an invariant-set, barrier, reachability, or runtime-shield certificate. At history h, the certificate induces actions that are valid for every plausible state:

$$K_\Gamma(h) = \{u \in \mathcal{U} : u \text{ satisfies } \Gamma \text{ for all } x \in X_{\mathcal{A}}(h)\}. \tag{4}$$

We call $K_\Gamma(h)$ the *certified kernel*. Severe-attack safety filters construct or act within such kernels for fixed controllers. Online learning additionally requires admission of the updated action map produced by U.

Definition 1 (Certified online learning)**.** *A mechanism is a certified online learner at history h if it accepts a certificate for $\pi_{\theta_{t+1}}$, where $\theta_{t+1} = U(\theta_t, h)$. The accepted certificate asserts that the next certified action is sound for every state in $X_{\mathcal{A}}(h)$.*

Definition 2 (Finite-horizon check)**.** *Let $\mathcal{H}^{\tau}_{\mathcal{A}}(h, \pi)$ denote the attacked histories reachable within the next τ steps from h under policy π. A finite-horizon check is sound for its declared horizon if acceptance implies that $\pi(h')$ satisfies Γ for every $h' \in \mathcal{H}^{\tau}_{\mathcal{A}}(h, \pi)$ and every $x \in X_{\mathcal{A}}(h')$. It makes no claim about histories outside this set.*

Definition 3 (Sound lifecycle certificate)**.** *A certificate accepted for π_{θ^+} at history h over attack family $\mathcal{A}$ is* sound *when it asserts, for every attacked history h' reachable after h under π_{θ^+} and every $x \in X_{\mathcal{A}}(h')$, that $\pi_{\theta^+}(h')$ satisfies Γ. An acceptance rule is sound for a contract when acceptance implies the safety property that contract requires.*

Definition 4 (Learning-freeze frontier)**.** *For a family of histories parameterized by attack strength or ambiguity radius, the learning-freeze frontier is the boundary at which $K_\Gamma(h)$ becomes empty, the candidate updated action leaves $K_\Gamma(h)$, or the candidate parameter leaves the corresponding certified set $\Theta_\Gamma(h)$. Beyond this boundary, a sound certified learner must reject the certificate, freeze the update, or obtain extra trusted state information.*

2.3 Assurance-Contract Failure Modes

Four failure modes arise at different points in the assurance lifecycle, and three of them are already recognized. *Nominal-state certification* evaluates the certificate at an estimate $\hat{x}(h)$, so under FDI an accepted action may be unsafe for another state in $X_{\mathcal{A}}(h)$; severe-attack safety filters address it. *Action-only filtering* projects the applied action into $K_\Gamma(h)$ but leaves the updated action map a separate object, so the certificate can cover the actuator channel while $\pi_{\theta_{t+1}}(h) \notin K_\Gamma(h)$. *Uncertified learning after rejection* occurs when an empty kernel forces a fail-safe action while an update computed from the same attacked history proceeds anyway.

The fourth mode is the subject of this paper. Under *protected adaptation with unsafe deployment*, an adaptation shield keeps every data-collection transition safe. Safety of those executed transitions does not establish the behavior of the unfiltered raw policy that will later act alone. Raw release therefore requires evidence for the subject receiving action authority. Corrupted reward records are not needed to reach this mode; they widen it by steering the trusted update rule toward a raw policy that fails after retirement while protected execution remains safe.

These modes require separate metrics: accepted certificates whose applied action is unsafe for some plausible state, accepted certificates for which the updated raw policy leaves the kernel, and nontrivial updates after certificate rejection. End-to-end studies add delayed deployment violations, admission coverage, paired clean/poison outcomes, update fraction, intervention, reward, and certificate runtime. Useful learning additionally requires paired task improvement over always-freeze.

3 Threat Model

3.1 System Roles and Deployment Contracts

The defender deploys an online-updated controller π_{θ_t} and a certificate mechanism; a severe-attack-aware estimator or

Table 1: Evaluated write boundary and the read authority the evaluated attack consumes. "Trusted" means outside attacker write authority, not formally verified.

Object	Defender assumption	Write access	Read access
Runtime state, action, plant	Authenticated control path	No write	Logged copy
Baseline, selector, certificate	Trusted runtime code/state	No write	No read
Transitions and learner code	Trusted after collection	No write	Read
Reward record at ingestion	No origin/semantic check	Bounded scalar write	Read
Parameters and optimizer state	Written only by updater	No write	Read
Held-out states and release rule	Fixed, honest evaluation	No write or selection	No read

filter returns a plausible-state set $X_{\mathcal{A}}(h_t)$ and certified action kernel $K_{\Gamma}(h_t)$. A trusted state anchor, backup controller, or external fail-safe may also be available, but each is an explicit architectural assumption and cost.

The evaluation instantiates four contracts. *Permanent raw release* removes runtime authority after a one-time five-step check: protected commissioning is followed by snapshot export and promotion to a lower-latency control target, which avoids permanent computation, intervention, and dependence on a backup service. *Commit admission* certifies a candidate snapshot on a declared state set and horizon before raw release. *Resident predictive authority* repeats the raw-policy check online and can transfer control to the trusted baseline. The check supplies evidence about the raw policy, not about the baseline response from the handoff state; the latter is a separate baseline-handoff obligation. The *one-step resident kernel* applies a current-state action filter and serves as a negative control. A sound filter that remains online instead certifies the composition $F \circ \pi_{\theta^+}$. Each name denotes a distinct evidence scope and authority contract, and Section 5 measures their tradeoffs.

3.2 Split-Trust Learning Pipeline

The least-privilege model separates a high-integrity runtime-assurance plane from an asynchronous learning-data plane. The runtime plane owns the authenticated plant state, action selector, actuator interface, trusted baseline, and certificate outcome. The learning plane consumes recorded transitions and scalar rewards through telemetry, a historian, a replay buffer, or an external key-performance-indicator (KPI) interface. OT guidance recognizes this separation and the exchange of historian data across zones [6, 7]. The benchmark models the resulting interface without assuming a particular product.

Table 1 states the evaluated boundary in both directions. The split-trust argument constrains write authority; the read column records what the evaluated attack rule consumes, including the learner state its edits are computed against, which is stronger than writes alone (Section 8.1). The attack begins after compromise of a principal with bounded reward-field write access, which permits post-computation, pre-update mutation by a replay writer or broker. A compromised reward producer is equivalent when it has the same bounded output authority; in that case source authentication establishes origin but not reward semantics.

Applicability condition. The learning-data plane is deliberately reachable: historians, ingestion brokers, replay buffers, and KPI services exist so that offline consumers can read and enrich operational records, and OT guidance treats cross-zone historian exchange as an expected data flow [6, 7]. A principal on that path holds bounded write authority over a scalar field long before it holds any authority over the control path, which is why we grant the attacker nothing else. The channel is only interesting when the runtime plane cannot recompute the reward from authenticated transitions. That is the case when reward depends on delayed quality labels, operator feedback, manual disposition codes, or an external KPI the runtime plane does not observe. It is not the case for rewards that are deterministic functions of authenticated state and action: there, trusted recomputation removes the channel outright, as our frozen integrity audit confirms (Section 5). Both simulation platforms expose deterministic rewards, so trusted recomputation is available in the experiments. We use them to isolate the consequences of the reward-ingestion interface, not as evidence that these benchmarks inherently lack recomputation. The deployment attack surface is limited to otherwise equivalent interfaces whose reward semantics cannot be reconstructed inside the high-integrity plane.

3.3 Attack Channels and Capabilities

The general history model permits two attack channels. Observation FDI a_t changes the state semantics of the history and enlarges $X_{\mathcal{A}}(h_t)$. Update-data corruption b_t modifies learning signals and can redirect $U(\theta_t, h_t)$ with trusted state observations. Either channel can alter the next action map.

The end-to-end learner experiments use the narrower reward-record channel. The white-box, batch-aware attacker observes the current learner and adaptation batch, but not the held-out deployment-state choice. It modifies only scalar rewards before an ordinary policy-gradient update, subject to a declared per-step L_∞ budget. It cannot write any object marked "No write" in Table 1. The adaptation selector observes each proposed action before the plant step and retains physical authority during data collection.

3.4 Evidence Transfer Across Authority Transitions

The contracts above differ in their evidence obligations. We represent the evidence used by an assurance decision as

$$E = (s, a, p, i, \tau). \tag{5}$$

Here s, the *subject*, is the raw policy π_θ or the assured composition $F \circ \pi_\theta$; a, the *authority*, identifies the resident module that constrains applied actions, or its absence; p, the *provenance*, records whether learning signals came from a trusted or an adversarial update process; i, the *information*, describes the state, observation, belief, or filtered action available at decision time; and τ, the *horizon*, is the interval over which the evidence bounds behavior.

A contract transition $C_{\text{train}} \to C_{\text{deploy}}$ may change one or more components, and evidence transfers only after every changed component is re-established under C_{deploy}. Evidence about the composition does not bound the raw policy, and a raw-policy audit does not certify the composition (*subject*). When a resident module masked unsafe proposals, what was collected describes the masked process, not the raw policy that acts after the module is removed (*authority*). An adversarial update log breaks the identification between the trained policy and the trusted process the evidence was computed on (*provenance*). Deployment observations differ from the authority's information set, so behavior under one does not determine behavior under the other (*information*). Finally, a finite evaluation pass leaves the remaining horizon untested (*horizon*).

The evaluated contracts contain two authority transitions. Permanent raw release transfers control from the assured composition to the raw policy, so it changes subject, authority, and horizon. Resident predictive authority retains the monitor but may later transfer control from the raw policy to the trusted baseline. Its trigger observes predicted raw-policy behaviour; it does not establish that the baseline satisfies the safety property from the handoff state. This second transition changes the certified subject and requires evidence for the baseline handoff.

Proposition 1 (Transfer obligation)**.** *Let E be evidence collected under contract C_{train}, and let the transition $C_{\text{train}} \to C_{\text{deploy}}$ change a nonempty subset of the components of the evidence tuple. Evidence E does not by itself establish sound acceptance under C_{deploy}; each changed component's obligation must be re-established for that contract.*

The proposition supplies the security model used throughout the paper. Independent transfer evidence re-establishes obligations before authority removal [5]. Resident Simplex mechanisms preserve the authority component [1, 8, 9]. Carr et al.'s lifecycle supplies a published transition from shielded bootstrap to raw-policy authority [4]. Our case study adds adversarial update provenance to that transition, and the controlled study isolates the resulting mechanism.

3.5 Integrity Assumptions and Escape Conditions

The vulnerable configuration supplies neither origin integrity for the reward record nor trusted reward recomputation. Binding a reward, transition, action, producer identity, and time to an authenticated append-only record blocks post-ingestion mutation, and recomputation by a trusted principal completely blocks the evaluated channel whenever reward is a deterministic function of authenticated transitions. Under the applicability condition above, the system instead needs a trusted producer, redundant validation, or a detector; a signature applied to an already malicious value is insufficient.

These controls act at different boundaries: provenance and recomputation remove the upstream precondition, a task-aware detector can reject suspicious batches under its distributional and semantic assumptions, commit admission restricts which learned snapshot can be released, and resident predictive authority adds a downstream handoff whose safety depends on the trusted baseline and the state at which it receives control. In the frozen audit, trusted recomputation achieved true- and false-positive rates (TPR/FPR) of 1.000/0 and simple task-aware checks achieved TPR above 0.90 and FPR at most 0.036 against the attack rule we first evaluated. Only recomputation closes the evaluated channel: an attacker that respects the task-aware predicates by construction retains deployment harm, and the resident handoff can still be followed by a violation (Section 5.4). Those checks bound one attack rule rather than the channel or the downstream handoff.

3.6 Security and Availability Goals

We separate four attacker or failure objectives. An *unsafe certified action* occurs when the mechanism accepts an applied input that is unsafe for some state in $X_{\mathcal{A}}(h_t)$. A *stale certified policy* occurs when the current applied input is filtered but the accepted raw updated action map leaves the kernel. *Uncertified learning* occurs when a nontrivial update continues after certificate rejection. *Forced freeze* occurs when the attacker expands the plausible set or redirects the update until sound learning must stop or re-anchor. The defender also measures the availability metrics of Section 2.3; useful learning requires paired task improvement over always-freeze in addition to physical safety.

Severe-attack filters, secure estimators, and fault-tolerant control-barrier function (CBF) methods can preserve safety when their assumptions keep $K_\Gamma(h_t)$ nonempty [14–16]. Out of scope are service exploitation, compromise of trusted recomputation, and attacks on the authenticated runtime state or baseline controller; Section 8.1 states the remaining validity boundaries.

4 Contract-Aware Update Analysis

This section develops the update conditions used by the empirical studies. The current- and future-history gates distinguish action filtering from certified learning and provide the obligations against which authority transitions are evaluated. A reward-influence summary then records how a bounded reward edit can move a linear release-gate margin within one policy-gradient batch; the evaluated multi-batch attack remains the fixed target-shaping rule described in Section 5. All proofs are deferred to Appendix A.

4.1 Current- and Future-History Gates

The certificate condition has two levels. The current-history condition requires the updated action at the observed history to remain certified. The lifecycle condition extends this requirement to future attacked histories reachable after the update.

Lemma 1 (Current-history gate)**.** *Let $M = (\pi_\theta, U, \mathsf{Cert})$ be a history-dependent online controller. For an attacked history h, let $X_{\mathcal{A}}(h)$ be the plausible-state set and $K_\Gamma(h)$ its certified kernel. If M accepts a sound certificate for $\pi_{\theta_{t+1}}$, where $\theta_{t+1} = U(\theta_t, h)$, then*

$$\pi_{\theta_{t+1}}(h) \in K_\Gamma(h). \tag{6}$$

Consequently, if $K_\Gamma(h) = \emptyset$, then M cannot both accept the certificate and allow a nontrivial online update at h.

An action filter that returns a certified input at h does not certify the update that produced it. The filtered actuator channel can satisfy the certificate while $\pi_{\theta^+}(h) \notin K_\Gamma(h)$, so the updated learner remains uncertified; Appendix A states this as Proposition 3.

These gates are local, but a policy update also determines actions at future histories. Let $\mathcal{H}_{\mathcal{A}}(h, \theta^+)$ denote the attacked histories reachable after h under π_{θ^+} and attack family $\mathcal{A}$, and define the certified parameter set $\Theta_\Gamma(h) = \{\theta : \forall h' \in \mathcal{H}_{\mathcal{A}}(h, \theta),\ \pi_\theta(h') \in K_\Gamma(h')\}$.

Theorem 1 (Future-history lifecycle gate)**.** *Let $\theta^+ = U(\theta_t, h)$ be a candidate online update. Suppose M accepts a sound lifecycle certificate for π_{θ^+} from history h over attack family $\mathcal{A}$. Then*

$$\theta^+ \in \Theta_\Gamma(h), \tag{7}$$

equivalently,

$$\forall h' \in \mathcal{H}_{\mathcal{A}}(h, \theta^+), \quad \pi_{\theta^+}(h') \in K_\Gamma(h'). \tag{8}$$

If membership fails for a reachable h', accepting the update creates a stale certified policy; if $K_\Gamma(h') = \emptyset$, certified learning must reject, freeze, roll back, or obtain trusted information that changes $X_{\mathcal{A}}(h')$.

Proposition 2 (Linear-policy parameter gate)**.** *Consider a finite abstraction of future attacked histories indexed by $z \in \mathcal{Z}$ and a linear policy $\pi_\theta(z) = \theta^\top \phi(z)$. Suppose each certificate produces an interval kernel $K_\Gamma(z) = [\ell_z, u_z]$. The lifecycle condition over $\mathcal{Z}$ is then equivalent to*

$$\theta^\top \phi(z) \le u_z, \qquad -\theta^\top \phi(z) \le -\ell_z, \qquad \forall z \in \mathcal{Z}. \tag{9}$$

Optional box constraints on θ preserve a convex polytope. Rejecting candidates outside this set, or projecting onto it, is sound for the finite abstraction.

Proposition 2 is exact only on its listed grid. Appendix B states the coverage, regularity, model-error, and margin assumptions under which it lifts to continuous reachable histories, together with a plausible-set monotonicity result, the kernel instantiations for invariant and barrier certificates, and a minimal counterexample separating fixed filtering from certified learning.

4.2 Reward Influence on a Release Gate

Editing a reward after collection leaves the states, actions, and score rows of a batch unchanged, so a bounded edit produces gate-directed parameter motion inside that batch. Appendix C makes the resulting influence exact under global Euclidean gradient-norm clipping, giving a per-batch certificate that no admissible edit crosses a release halfspace. Its consequence is negative and it is the reason the evaluated attack must accumulate: the result is confined to one batch and does not explain the multi-batch attack we evaluate, which is the fixed target-shaping rule of Section 5. Coordinatewise parameter clipping remains outside the exact support computation.

4.3 Certificate-Gated Update and Deployment Contracts

The resulting update rule first obtains $X_{\mathcal{A}}(h)$ from a severe-attack-aware estimator or safety filter and computes $K_\Gamma(h)$. An empty kernel rejects the certificate and freezes the update. Otherwise the mechanism accepts, projects, constrains, or rejects the candidate according to the updated action's kernel membership. For linear policies the admission test is the convex halfspace system of Proposition 2; a grid certifies only its listed points unless the cover, regularity, and model-error premises of Appendix B justify a robust margin. We call this enforcement layer LifecycleGate.

The empirical contracts use the finite-horizon check of Section 2, not the sound lifecycle certificate defined there. A one-time release evaluates its declared horizon at h and then executes π_{θ^+} without further evaluation. A resident mechanism repeats the same check at subsequent histories, thereby extending observation through a receding horizon. Repetition does not certify the baseline controller invoked after rejection. A sound lifecycle certificate would instead cover every future

history required by the contract, or supply a valid terminal or recovery argument; a controller handoff would additionally require evidence for the receiving controller from the transfer state. Section 5.1 measures these distinct obligations. Commit admission certifies a candidate on a declared state set and horizon, backtracking along the connected segment from the last admitted snapshot to exclude disconnected safe islands. Its costs are coverage and model-mismatch risk; resident mechanisms instead incur intervention, rejection, and online cost. An empty plausible-state kernel, an uncertified snapshot, model failure, or solver failure triggers a fail-closed freeze or rollback, and a trusted anchor can resume learning by changing the justified plausible-state set at an explicit infrastructure cost. Because membership is enforced independently of the task objective, a safe update may be neutral or harmful: accepted parameter distance measures update availability, whereas paired task outcomes against always-freeze measure learning utility under benign change.

5 Controlled Mechanism Study

We first isolate the authority-transition mechanism in Safe-Control-Gym [17]. The study compares permanent raw release with resident authority, bounds the resident check's lookahead, and tests sign/envelope-constrained attacks. Section 6 then examines the same transition in a published lifecycle; secondary mechanism and cost audits appear in the appendices.

5.1 Prospective Contract-Separation Audit

All results in this subsection assume the reward-ingestion interface in Table 1. Development grids selected target $(18,-5)$ using five-step acceptance and 120-step harm. Before the sequential V3 audit, we fixed the target, learner, state pools, stop criterion, and bounded perturbation $2\tanh(\varepsilon_t(\mu_t^\star-\mu_t))$ on the development run indexed by seed 2070; runs 2071 and 2072 were then evaluated under the unchanged protocol.

The adaptation filter falls through to the learner proposal if its action kernel is empty, so safe adaptation is measured here rather than guaranteed by that implementation path. Across the reported fixed-target runs, `rejected_action_steps` and adaptation constraint violations were both zero, while the filter intervened on 96 proposals. Every executed training transition therefore remained within the measured constraint set.

The gap without an attacker. A disjoint clean cohort contains 20 learner/evaluation seed pairs. Each clean-trained snapshot was evaluated on 24 states accepted by its own clean five-step CasADi check, giving 480/480 accepted pairs. Permanent raw release failed on 10/480 pairs, distributed over five independent training runs, while resident predictive authority failed on 0/480 and switched on 11 pairs. The 10 raw-only and zero resident-only discordances give a two-sided exact paired $p = 0.00195$, conditional on the 20 trained snapshots. At the training-run level, 5/20 runs contained at least one raw-release failure (Wilson 95% CI $[11.2\%, 46.9\%]$). The pair-level raw rate was 2.08% (Wilson 95% CI $[1.14\%, 3.79\%]$), against 0/480 for residence (upper 95% bound 0.79%).

Table 2: Contract separation after initial five-step acceptance. V5 uses clean-snapshot acceptance; V4 uses the poisoned-snapshot acceptance set locked for its three original arms. The independent audit uses poisoned snapshots. Rows are paired within a trained snapshot but training runs are the independent replicates. A switch transfers control to the trusted linear-quadratic regulator (LQR) baseline.

Contract	Failures	Authority event
V5 clean cohort (480 pairs; 20 training runs)		
Clean permanent raw release	10/480	none
Clean resident predictive authority	0/480	11 switches
V4 fixed-attack confirmation (120 pairs; 5 runs)		
Clean permanent raw release	2/120	none
Clean resident predictive authority	0/120	2 switches
Poisoned permanent raw release	27/120	none
Poisoned resident predictive authority	0/120	27 switches
Independent audit (71 accepted states, poisoned snapshots)		
Permanent raw release	19/71	none
One-step resident kernel	14/71	79 int.; 41 empty
Resident predictive authority	0/71	19 switches

All 72 poisoned pairs passed the five-step check. Permanent raw release then failed on 23/72 (11, 5, and 7 across the runs), versus 2/72 for paired clean release and 0/72 under resident predictive authority. The 21 poison-only and zero clean-only discordances gave two-sided exact paired $p = 9.537 \times 10^{-7}$; every switch led the paired raw failure by four or five steps. The frozen protocol was then evaluated once on five new learner/evaluation pairs. Poisoned raw release failed on 27/120 pairs across all five runs, versus 2/120 clean and 0/120 resident; resident authority switched before every poisoned failure (25 poison-only, zero clean-only, $p = 5.960 \times 10^{-8}$).

V4 remains a harness check: resident authority removed its two clean failures, and replay reproduced all 27 poisoned switches with zero failures. V4 used poisoned-snapshot acceptance; V5 used clean-snapshot acceptance, so their denominators answer different conditional questions and are not pooled. Pairs sharing a learner are repeated measurements: exact tests summarize within-snapshot discordance, while affected training runs report replication. The independent one-step audit used disjoint evaluation pools; 71/72 states passed, with shared state–snapshot keys (Table 2).

How much lookahead a retained check needs. In the locked

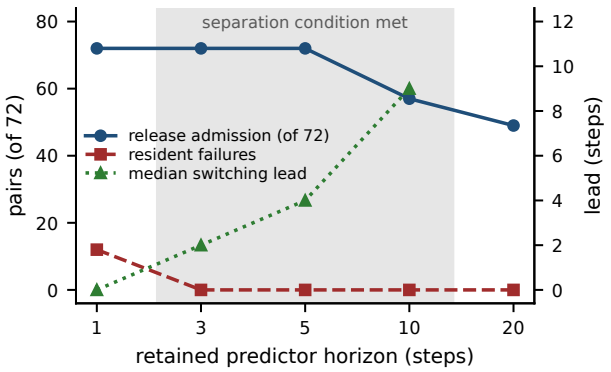

Figure 2: The retained check's horizon defines a usable interval, not a threshold. Pooled V3 cohort, 72 snapshot–state pairs, with attack, snapshots, and deployment states fixed; shading marks the horizons meeting the predeclared separation condition. At horizon 1 the retained check switches with zero lead and still leaves 12 failures; at horizon 20 the release check rejects so many candidates that no poisoned failure remains to separate.

72-pair V3 sweep, horizons 1, 3, 5, 10, and 20 admitted 72, 72, 72, 57, and 49 pairs. Median switching lead grew from 0 to 2, 4, and 9 steps through horizon 10; resident failures fell from 12 at horizon 1 to zero at horizons 3–10. Horizon 20 was safe but rejected enough candidates to eliminate the separation cohort. Thus switching lead time bounds the lower edge and affordable admission loss the upper edge (Fig. 2).

In the independent audit, permanent raw release failed on 19/71 and resident predictive authority on 0/71 after switching four steps early. The one-step resident kernel failed on 14/71 despite 79 interventions and reached an empty kernel 41 times. Repeated prediction followed by baseline handoff contained the paired raw failures in this cohort; one-time authorization and one-step filtering did not.

5.2 Reward-Influence Geometry and Optimizer Boundary

Reconstructing the implemented gradient has error 2.78×10^{-17}, and the largest discrepancy between a second-order cone program (SOCP) witness and its computed support is 1.07×10^{-9}. On all 36 poisoned V3 batches the homogenized formulation yields verified support witnesses on 22/36, against 0/36 for the direct one. Exact batch support nevertheless did not explain the successful multi-batch attack: the physical experiment evaluates the fixed target and bounded attacker without establishing joint gate optimality of the tanh rule. Appendix D reports the full audit and its parameter-clipping boundary.

5.3 Cross-System Reward-Record Poisoning

We next tested the lifecycle failure with updates produced by an online learner: a Gaussian residual actor on a trusted controller, with standard reward-to-go REINFORCE gradients computed from PyBullet transitions. A backup action filter protects the adaptation trajectory, and deployment then audits the raw learned snapshot. Training, certificate, calibration, and deployment states used disjoint run indices; all mechanisms within a training realization shared paired state sets; and attack directions and hyperparameters were fixed on a development realization before the three-run audits. The exact tests operate at the paired deployment-state-block level conditional on the three learned snapshots, so their p-values summarize within-snapshot discordance, and the interval estimates have the same conditional scope.

Table 3 reports the paired outcomes for both systems. On cartpole, the clean learner violates 1/188 admitted rollouts and the poisoned action-only learner violates 38/188. There are 37 paired poison-only failures and no clean-only failures (two-sided exact paired $p = 1.455 \times 10^{-11}$). Both commit admission and always-freeze have zero observed violations.

On two-dimensional quadrotor, clean learning has 0/72 violations while reward-only poisoning causes 68/72, all delayed, across all three training realizations (68 poison-only, zero clean-only, two-sided exact $p = 6.776 \times 10^{-21}$). Every one of those failures is delayed past adaptation, so protected execution reports nothing about any of them: the separation follows the authority transition, not the platform. Commit admission and always-freeze again have zero observed violations, and the gate retains 0.55, 0.40, and 0.70 of the poisoned pending updates, which establishes update availability under attack; benign learning utility is measured separately.

Two adjacent checks retained the result. A learned value baseline caused 40/126 poisoned versus 2/126 clean raw failures, with commit and freeze at 0/126; this is a robustness check within the Monte Carlo policy-gradient family. With a pole-angle bias of radius ρ, poisoned counts were 44/126 and 61/126, versus 4/126 and 3/126 clean, at $\rho = 0.02$ and 0.04; commit and freeze remained zero. Appendix E reports the protocol and the near-singleton reference.

5.4 Sign/Envelope-Constrained Duration Stress Test

The fixed attack above crosses zero reward, so a task-aware nonpositive-reward check detects it. To test the check as an attacker constraint rather than as a post hoc detector, we projected the same target-shaping edit into the intersection of its L_∞ budget and the scalar envelope calibrated on the clean 12-batch development run. If the authenticated true reward was already too far outside that envelope for any budget-feasible edit to enter it, the attacker made no edit. A batch-mean constraint was enforced by uniform shrinkage. We varied

Table 3: Paired reward-record poisoning and snapshot admission. "Commit" denotes finite-horizon admission; permanent mechanisms use the common-state audit in Fig. 13. Raw-snapshot rewards after failure are descriptive, not utility comparisons.

System	Mechanism	Learner seeds	Rollouts	Violations	Mean reward
Cartpole	Clean REINFORCE	3	188	1	−0.197
Cartpole	Poisoned action-only	3	188	38	−1.269
Cartpole	Poisoned commit	3	188	0	−0.102
Cartpole	Always-freeze	3	188	0	−0.117
Quadrotor	Clean REINFORCE	3	72	0	−0.030
Quadrotor	Poisoned action-only	3	72	68	−0.170
Quadrotor	Poisoned commit	3	72	0	−0.060
Quadrotor	Always-freeze	3	72	0	−0.029

only the protected bootstrap duration over 12, 24, and 48 batches on five new learner/evaluation seed pairs. Clean, unconstrained, and constrained conditions used paired run indices and deployment-state generation.

Table 4 changes both the detectability and contract outcomes. The sign/envelope-constrained attack produced no known-sign flags at any duration. It produced no scalar-envelope flags at 12 or 24 batches and 2/240 at 48 batches, fewer than the 6/240 flags from clean training. The frozen envelope therefore did not separate attack from benign duration drift. Attack effect also did not remain halved: constrained raw-release failure was 17/120 at 12 batches and 46/116 at 48, compared with 36/119 and 54/117 for the unconstrained positive controls. Trusted recomputation still detects every nonzero edit, but it is unavailable by the applicability condition when reward semantics depend on an external KPI.

At 48 batches, resident predictive authority failed on 34/116 constrained pairs, concentrated in two of the five independently trained controllers with 17 failures each; clean resident authority remained at 0/120. All 34 resident failures occurred after transfer to the trusted LQR, with violation following the switch by a median of one step and at most two. Paired accounting gives a more precise comparison: 27 pairs failed under both contracts, 19 failed only under raw release, seven failed only under resident authority, and 63 failed under neither. Among the 27 pairs that failed under both, the switch preceded the raw-policy failure by a median of four steps (range four to five). The seven resident-only failures have no paired raw-policy failure from which to define such a lead.

Repeated raw-policy checks therefore prevented 19 failures observed under raw release, but they did not establish the safety of the controller receiving authority. Because the experiment does not characterize a certified recovery set for the LQR, the data do not determine whether a different switch time would have prevented a violation. The supported conclusion is that warning evidence about the raw policy is not evidence for the baseline handoff.

The earlier frozen audit remains useful only for scoping controls. Trusted recomputation achieved TPR/FPR 1.000/0, while known-sign and scalar-envelope checks detected above 90% of the original sign-violating rule (Table 7). An in-loop known-sign gate likewise reduced that original rule from 39/126 raw-release failures to 0/126. The duration stress test shows why neither result establishes channel closure: a constrained attacker respects the same predicate by construction.

5.5 Deployment Criteria and Measured Boundaries

The evidence supports three deployment criteria. Permanent raw release must re-establish every premise changed when the assured composition relinquishes authority (Proposition 1). Resident authority must pair its raw-policy warning rule with independent evidence that the baseline can accept the handoff states. Finally, adaptation should provide sufficient task benefit to justify the transition risk and the coverage, intervention, latency, and recovery costs of the selected contract. The horizon sweep measures warning time for raw-policy failures; it does not measure a certified baseline recovery margin. The duration study's paired outcomes show why this baseline-handoff obligation cannot be replaced by the trigger alone.

The evaluated benign settings provide no measured adaptation benefit. Under a 0.003 N actuator bias, freeze, clean adaptation, and LifecycleGate were all violation-free, but neither clean adaptation nor the accepted updates improved a paired rollout over freeze. Across the unified 24-block frontier, no evaluated mechanism improved on freeze. For the studied deployments, the observed task outcomes provide no utility basis for accepting the additional transition risk.

Appendices E, H, and I report the secondary infrastructure and measurement audits: trusted anchors, PPO, certificate coverage, model mismatch, admission, runtime latency, and reward cost. In the unified frontier, poisoned raw release failed on 23/24 paired blocks, while clean, commit, freeze, permanent shield, and the official linear model predictive safety certification (MPSC) mechanism all had zero observed failures. The CasADi gates remain sampled finite-horizon certificates,

Table 4: Duration stress test over five training runs per condition. Raw and resident columns report marginal physical-failure counts among five-step accepted pairs; paired accounting for the 48-batch constrained condition is reported below. Known-sign and scalar-envelope columns count flagged training batches. The constrained attacker respects both predicates whenever a budget-feasible edit exists; flags can still arise from the unedited true reward leaving the frozen 12-batch envelope.

Batches	Accepted	Condition	Raw failures	Resident failures	Sign flags	Envelope flags
12	120	Clean	4	0	0/60	0/60
12	120	Constrained	17	0	0/60	0/60
12	119	Unconstrained	36	0	56/60	56/60
24	120	Clean	6	0	0/120	1/120
24	120	Constrained	15	0	0/120	0/120
24	120	Unconstrained	16	0	89/120	96/120
48	120	Clean	9	0	0/240	6/240
48	116	Constrained	46	34	0/240	2/240
48	117	Unconstrained	54	38	144/240	160/240

not continuous-state invariant-set proofs, and realized state violations and pre-clip saturation are reported separately.

6 Third-Party Shield-Retirement Case Study

This section examines the same evidence boundary in an independently published lifecycle that retires runtime authority. Carr et al. [4] (AAAI 2023) train a policy-gradient agent behind a Storm belief-support shield in a partially observable gridworld, then remove the shield suddenly or smoothly and compare the released policy with retained-shield and no-shield controls. We use the official public implementation and modify only learner-visible reward records: the attacker writes $r' = r + \xi$, with $|\xi| \leq \delta$ per record, before the trusted update. States, observations, shield semantics, dynamics, learner code, hyperparameters, and evaluation follow upstream, and the artifact itemizes every instrumentation and attack patch. REINFORCE training runs for 5000 episodes with retirement at episode 1000; the TF-Agents proximal policy optimization (PPO) pipeline retires the shield after 10^5 environment steps.

Lifecycle fidelity. On the obstacle domain (N=6), the clean condition reproduces the qualitative ordering of Carr et al.'s Table 1 for REINFORCE: shield retained 0/0 violations during/after training, smooth 326/675, sudden 3277/3784, and no-shield 3798/3784. Shield $<$ smooth $\ll$ sudden $\leq$ no-shield is the ordering reported by Carr et al., establishing the lifecycle fidelity of the instrumented implementation.

Causal isolation at the retirement boundary. We use two locked poisoning scopes. Under *bootstrap-only* scope, reward-record mutation is active only while the shield enforces the action mask, stops at retirement, and all subsequent learning is clean. Under *full-phase* scope it stays active for the whole run, including post-retirement learning; Appendix F reports its batteries. Unless stated otherwise the results below use bootstrap-only scope. We evaluate the raw policy at the switch instant before any unshielded learning (1000 episodes) and again after the full budget (5000 episodes). Each independently trained clean/poisoned pair is one training-level replicate; configured seeds serve only as run indices, and a pair is *provenance-complete* when both its clean and its poisoned endpoint trace to the same locked configuration on the same host. A pair is *positive* when the poisoned at-retirement rate exceeds its clean pair by at least 0.15 and the paired McNemar exact test gives $p < 0.01$; all other outcomes are *non-positive*, and *protective* denotes a poisoned rate at least 0.15 below clean. The significance clause never binds at this episode count: every observed difference of 0.15 or more had $p < 10^{-29}$, and every pair with $p \geq 0.01$ differed by less than 0.02.

Table 5: Retirement-boundary causal isolation on the third-party lifecycle (obstacle, REINFORCE, sudden). Each row is a provenance-complete paired training run. Poisoning stops at retirement, and all later training is clean. At-retirement evaluates the raw policy at episode 1000; final evaluates it at episode 5000. Rates are violation fractions.

	at retirement		final	
configured seed	clean	poisoned	clean	poisoned
1	0.216	0.777	0.236	0.235
2	0.484	0.763	0.234	0.751
3[a]	0.241	0.242	0.747	0.228

[a] A routine integrity sweep over all runs found the earlier run-index-3 clean at-retirement record byte-identical to a summary file from a separate verification run; the same sweep cleared rows 1 and 2. Both endpoints of row 3 were therefore regenerated in a fresh namespace, on a different host from rows 1 and 2, so row 3 is a same-host pair drawn from a different host. The archived local poisoned realization (0.748 at retirement) is excluded because pairing it with the regenerated clean endpoint would cross hosts. The exclusion rule was recorded on 2026-08-17, after the 2026-08-14 protocol lock.

Two of the three provenance-complete pairs had higher raw-policy violation rates at retirement under poisoning, by factors of 3.6 and 1.6 (Fig. 3). The fresh third pair was unchanged (0.241 $\rightarrow$ 0.242). Physical violations remained zero

throughout protected execution. The positive outcomes located latent policy damage before unshielded learning, and the fresh pair showed the fixed attack can also fail in a low-disagreement retirement state. After 4000 episodes of clean post-retirement learning, one positive pair recovered (0.235 vs. clean 0.236), one remained worse (0.751 vs. 0.234), and the fresh non-positive pair reversed (0.228 vs. 0.747). We denote clean raw-policy/shield disagreement at retirement by D_{ret}. In the two positive pairs, D_{ret} was 0.025 and 0.047 and rose to 0.113 and 0.109 under poisoning. The fresh non-positive pair also had low clean disagreement ($D_{\text{ret}} = 0.013$), separating available headroom from attack success.

Learner and budget boundary. The same contrast attack was also evaluated on the third-party PPO and off-policy soft actor-critic (SAC) pipelines, across attack budgets and attack shapes, and on a second gridworld of the same suite. It raised post-removal violations in individual PPO and SAC training realizations without producing reliable success within either family. Its dose response depended on the realized retirement state rather than the nominal seed. The second environment reproduced the lifecycle ordering while remaining realization-dependent under attack. Appendix F reports these batteries in full.

Escape condition. Under full-phase poisoning, a retained shield kept physical violations at zero before and after training in all three paired runs. The corresponding sudden-retirement runs exposed final violation fractions of 0.490, 0.752, and 0.744 after removal. The retained shield's winning-region mask guarantees zero violations while a safe masked action exists. Full-phase contamination never produced an empty mask in these runs, so retained authority remained available.

Interpretation. Carr et al.'s workflow supplies the lifecycle component: protected bootstrap followed by retirement of the shield's action authority, evaluated under unpoisoned learning. The controlled study in Section 5 supplies the adversarial-provenance mechanism. Their composition exposes the evidence boundary at that authority transition. Absolute rates differ across instrumented and non-instrumented code versions because the unmasked-disagreement probe requires an independent belief tracker. Each clean/poisoned comparison is paired within one locked code version.

Training outcomes varied across platforms. A ten-run REINFORCE shield-on-only battery on Linux produced three positive low-clean-violation realizations ($0.240 \to 0.533$, $0.234 \to 0.744$, $0.229 \to 0.763$) and seven ceiling realizations with no added damage, whereas the fresh pair in Table 5 was non-positive despite low clean disagreement. The same configured seed and code can enter different trajectory modes across hosts or backends, so we treat each independently trained clean/poisoned pair as a replicate and use nominal seeds only as run indices. Episode-level McNemar tests describe outcomes conditional on a trained pair; the multi-run batteries provide the training-level replication evidence.

Across 53 locked paired runs spanning both scopes, all three learner families, and both domains, 12 of 20 runs with clean at-retirement violation below 0.5 were positive, whereas none of 33 ceiling runs was positive (Appendix F, Fig. 7). Low clean violation therefore marks available headroom but does not determine the outcome.

Corpus accounting. The 53-run susceptibility corpus contains one contrast configuration for each clean/poisoned training pair: ten pairs for each learner–scope combination in the obstacle domain, except that SAC has no full-phase battery, plus three REINFORCE isolation pairs in the avoid domain. Twelve comparisons met the positive rule, 20 had a rate difference at or below -0.15, and 21 lay between the two effect thresholds. In the sub-ceiling stratum, the corresponding counts were 12, 3, and 5; all 33 ceiling comparisons were protective or unchanged. Effects were therefore not uniformly harmful over the run-level corpus, while positive outcomes were confined to the sub-ceiling stratum.

The artifact also retains a 54-comparison obstacle-only configuration table. It replaces the three avoid-domain pairs with four exploratory full-phase REINFORCE comparisons at two attack budgets over two run indices. Because those configurations reuse run indices, that table is descriptive and is not used to estimate training-level prevalence or for an independence-based test.

Because clean training runs of this lifecycle enter bimodal trajectory modes, we calibrated the decision rule against clean-versus-clean replication: one of eleven same-version clean replicate pairs crosses the 0.15 threshold (Appendix F). A single paired run therefore can cross the effect threshold under clean replication. Training-level inference therefore relies on the multi-run batteries.

Disagreement instrumentation gives a second view of the obstacle-domain retirement state, not independent confirmation. Its reproducible exclusion result is that every retained positive outcome had clean $D_{\text{ret}} \leq 0.047$, non-positive outcomes extended to 0.111, and none of 32 high-disagreement runs was positive. Holding out a learner family preserved the exclusion for all 20 PPO rows, but all SAC rows were low-band and only 3/10 were positive, so low disagreement did not predict success. Two historical REINFORCE rows lack an executable deduplication rule; we therefore omit the corpus size, exact positive count, and discrimination statistic that depended on those membership decisions. The marker remains a post-bootstrap, obstacle-scoped exclusion condition, not an attacker trigger or validated classifier. Appendix F records the audit and band chronology.

7 Related Work

Assurance cases and change impact. That evidence stops supporting a claim once the system it was collected on changes is the founding problem of safety-case maintenance. Kelly and McDermid give the impact analysis that decides which legs of an argument a change invalidates [18], and dynamic

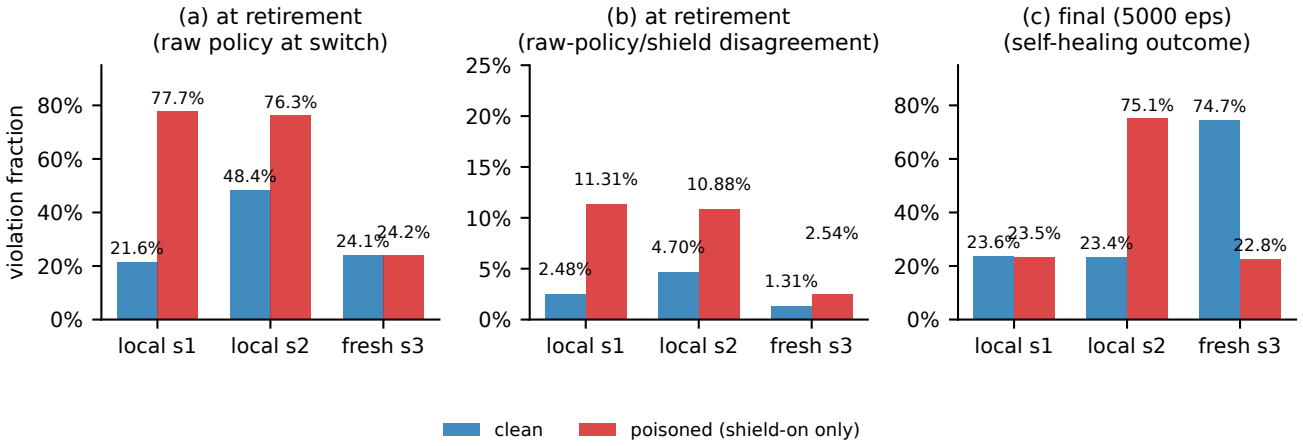

Figure 3: Causal isolation on the third-party lifecycle (REINFORCE, obstacle, sudden, provenance-complete paired runs). (a) At-retirement violation fraction of the raw policy; (b) at-retirement raw-policy/shield disagreement; (c) final violation fraction after clean post-retirement learning. Poisoning is confined to the shielded bootstrap phase. The run-index-3 bars show the fresh pair in Table 5.

safety cases carry it into operation so assurance is re-argued through life [19]. That literature asks whether engineers correctly re-argue safety after a known change. We instantiate that maintenance problem at controller-authority transitions: a release check and a later baseline handoff are separate change-impact events with distinct evidence obligations.

Runtime assurance and safe controller updates. Safety filters and shields form a design space, and we position against it as a whole [20]. Simplex architectures retain a decision module and a trusted baseline during online upgrades [1]; neural and barrier-based Simplex add online retraining, reverse switching, and switching conditions under bounded model error [8, 9]; and MPSC and runtime shields preserve recoverability or prevent unsafe actions [2,3,21]. All certify a resident composition; we consider a transition in which the raw policy later receives authority. Policy repair, safe projection, joint controller/certificate repair, robust conformal transfer, and shielded sim-to-real transfer instead establish evidence for the deployment object before use [5, 10–12, 22, 23].

Shield retirement and attacked states. Könighofer et al. [13] establish the non-adversarial half of this observation: policies trained under an online shield inflict more unshielded violations than conventionally trained ones, so shielding must be applied during execution. They further note that a finite-horizon shield needs sufficient lookahead, or the agent reaches states in which every action is unsafe; the 41 empty kernels our one-step control reached instantiate that situation. Our horizon sweep quantifies the tradeoff between warning time and release admission. The duration study then separates raw-policy warning from baseline handoff: 19 paired failures occurred only under raw release, whereas seven occurred only after resident transfer to the baseline. Because the experiment evaluates no recovery certificate, these outcomes do not locate the handoff states relative to a recoverable set. Carr et al. [4] study a partially observable bootstrap-to-retirement lifecycle and compare sudden with smooth shield removal under trusted learning signals. We reuse that published lifecycle as a third-party case study and add bounded reward-record poisoning during the protected bootstrap; the resulting experiment concerns evidence reuse across the same authority transition. Their trusted-signal setting serves as the clean control for the added data-integrity channel.

Secure estimation characterizes when sensor attacks can be detected or corrected [15, 24], and severe-attack CBF filtering constructs plausible-state sets and safe-action kernels when the state is not uniquely reconstructable [14]; our model consumes such a set and asks whether an adversarially updated action map stays inside its certified kernel. Related lines learn barriers or solve adversarial reach–avoid problems under trusted learning signals [25–30]. Table 11 in Appendix J places the closest work on the three axes this paper combines.

Policy and reward poisoning. Policy poisoning has been studied in batch reinforcement learning and control [31]; online attacks perturb rewards under an ℓ_∞ budget and adapt to the learner [32]; extensions cover policy-gradient and black-box deep learners, and targeted poisoning can teach a chosen policy [33–36]. None of this line places the learner behind a runtime shield.

Backdoors on shielded and classical controllers. Jiang et al. implant backdoors in formal-language-guided *safe* RL policies, so that a trigger state reached at runtime produces a safety violation [37]. This is the closest prior composition, and the difference matters: their trigger must survive a shield that is never retired, whereas our attacker plants nothing triggerable and the damage is inert until the deployment removes the monitor on the strength of a finite check. The exploited object is the release decision, not an input. BADControl poisons operational data in proportional-integral-derivative (PID) and

LQR loops to implant a vulnerability activated by a physical trigger [38], which shares the trigger structure without the learner or the shield. Where online reward-poisoning defenses model adaptive attacker–defender interaction [39], we separate provenance, trusted recomputation, detection, commit admission, and runtime containment.

Appendix C applies classical homogenization and second-order-cone feasibility [40–43] to the normalized image of a centered reward-to-go zonotope.

8 Discussion and Limitations

The evidence supports conditional exploitability at two authority transitions. Permanent raw release changes the subject, authority, and horizon of evidence collected during protected adaptation; the clean cohort exhibits this gap without an attacker. A later transfer from the monitored raw policy to the baseline changes the receiving controller, which the duration study shows to be a separate obligation.

In the Carr corpus, positive outcomes were confined to low retirement-state disagreement, but that region also contained non-positive outcomes. The 53-run accounting (12 positive, 20 protective, and 21 unchanged) supports a conditional, not uniform, attack effect. Controls map to different premises: trusted recomputation closes the reward channel, commit admission restricts raw-policy release, and resident authority requires evidence for the baseline handoff. Their coverage, intervention, latency, and reward costs remain an architectural tradeoff.

That tradeoff is justified only if adaptation provides paired task benefit over freezing the trusted controller. It did not in any evaluated benign setting: freeze was never worse than clean adaptation, and the locked benign audit stopped after 0/12 paired improvements. The resulting inference is limited to the present deployments, which provide no measured utility basis for accepting an additional authority-transition risk.

8.1 Limitations

The marker is descriptive, not an attacker trigger. Retirement-state disagreement is observed after the protected bootstrap, so it characterizes a realized deployment state; an attacker-side trigger would need earlier disagreement trajectories to predict the opportunity region. The high-band exclusion is reproducible, but two historical REINFORCE membership decisions were not recorded by an executable deduplication rule. We therefore report no corpus discrimination statistic, total row count, or exact positive count. Holding out one learner family also identifies only 3 of 10 SAC outcomes. The marker is an obstacle-scoped exclusion condition, not a validated classifier.

The formal results have bounded verification scope. Proposition 1 is a soundness statement about acceptance rules, not a claim about any particular policy. It organizes evidence obligations; no empirical result is derived from it. The current- and future-history gates state certificate obligations, and the linear parameter gate is exact on its finite abstraction; the continuous lift additionally requires a justified reachable-set cover, regularity and model-error bounds, and a robust margin, which the coverage audit probes without establishing continuous-state invariance. The exact reward-influence result covers a fixed standardized-advantage batch with global gradient clipping, leaving coordinatewise parameter clipping and multi-batch attack construction outside its scope.

Empirical generality is bounded to Monte Carlo policy gradients. The learned-baseline variant shares their return and attack channel, so it is not an independent family. The official Safe-Control-Gym PPO audit showed checkpoint-level redirection without stable physical failure, and on the third-party lifecycle PPO and SAC contained individual positive realizations without reliable success across paired runs.

We assume access, and do not estimate its prevalence. The evaluated attack rule also reads the learner state its edits are computed against (Table 1), so it assumes read access to the learner as well as one scalar write, which is stronger than the write alone. The task-aware checks detect the rule we first evaluated but not an attacker constrained to respect them, so only trusted recomputation closes the channel, and the applicability condition of Section 3 excludes it. At 48 batches the resident failures occur in two of five trained controllers. The study does not certify an LQR recovery set or identify which handoff states would be safe. These conditions limit claims about cross-architecture prevalence and deployment overhead.

9 Conclusion

A release test is not runtime authority: evidence collected under one contract does not certify a different controller or horizon. In the clean cohort, raw release failed on 10/480 pairs across five of 20 trained controllers; resident authority failed on none. Under the 48-batch sign/envelope-constrained attack, paired outcomes split 27 both, 19 raw-only, seven resident-only, and 63 neither. Every resident failure followed baseline transfer (median one step), so raw-policy warning did not certify the handoff. Carr et al.'s lifecycle showed the release transition in two of three paired runs, with conditional effects in the wider corpus. Trusted recomputation closes the reward channel; sound deployment must re-establish each authority-transition obligation and justify adaptation by task benefit, which the benign settings lacked.

Ethical Considerations

This paper studies an attack on safety-critical learning-enabled control. All experiments run in simulation using Safe-Control-Gym with PyBullet dynamics and a public third-party gridworld implementation. No physical plant, vehicle, deployed industrial system, human subject, or personal data was involved. Reported failures are simulated constraint violations used to compare deployment contracts.

The evaluated threat begins *after* compromise of a reward-ingestion principal in the asynchronous learning-data plane and grants only bounded scalar writes to reward records. Initial access, product-specific exploitation, compromise of trusted recomputation, and attacks on runtime control assets fall outside the artifact and evaluation. The experiments include evasion of calibrated task-aware checks and characterize the consequence of a least-privilege data-integrity gap for evidence inheritance at raw-policy release. The artifact contains no access-acquisition or field-targeting mechanism.

Stakeholders are operators of runtime-assured learning controllers, vendors of shielding and safety-filter tooling, the authors of the third-party lifecycle artifact we instrument, and the public exposed to the controlled plant. The main foreseeable harm from publication is that an attacker who already holds reward-ingestion access learns that protected adaptation can leave latent policy damage and that a baseline handoff can be followed by a violation. We judge the disclosure benefit to dominate: the exposure follows from documented assurance architectures, all experiments target public simulation artifacts, and the paper states countermeasures at both ends of the authority transition. Origin-bound reward records and trusted recomputation remove the upstream precondition; commit admission and permanent filtering limit downstream consequence; resident predictive authority does so only when its baseline handoff is independently supported.

The third-party lifecycle we instrument is a published research artifact released under an open license by its authors, used unmodified except for documented instrumentation and reward-record patches that the artifact itemizes. No vendor product, hosted service, or operational deployment was tested, so no vendor coordinated-disclosure process applies. The affected party is instead the artifact's authors. To preserve double-blind anonymity, communication and transfer of the instrumentation patches, paired result files, and manuscript are scheduled after the anonymity period.

Two Menlo principles determine the artifact scope. Beneficence: reproducibility requires the reward-shaping and sign/envelope-constrained scripts, so the artifact limits them to public simulation environments and includes no mechanism for obtaining the assumed ingestion access or targeting a fielded product. Respect for law and public interest: because the instrumented target is a licensed research artifact rather than a fielded system, we treated its authors as the party owed notice and handled disclosure as above.

Open Science

We support the USENIX Security open science policy and will make every artifact required to reproduce the reported results openly available. The artifact contains the experiment drivers for the controlled Safe-Control-Gym study and the third-party shield-retirement case study, and the row-level result files backing every table and figure, with source-linked provenance columns. It also records the exact commands, run namespaces, upstream revisions, model files, and dependency versions; the locked protocol documents recording the analysis chronology; and SHA-256 checksums for all locked files. Automated checks in the artifact audit three things: table provenance, claim locks, and manuscript format.

For anonymous review, the artifact is submitted as an anonymized archive through the submission system, so the review committees can access every component without contacting the authors. Upon acceptance we will publish the same artifact in a public repository with a permanent DOI under an open license, together with the third-party upstream revision pins required to rebuild the case-study environment. The artifact retains every scope boundary stated in Section 8.1, recorded with the data rather than removed.

References

[1] D. Seto, B. Krogh, L. Sha, and A. Chutinan, "The simplex architecture for safe on-line control system upgrades," in *Proceedings of the 1998 American Control Conference*, vol. 6, 1998, pp. 3504–3508.

[2] M. Alshiekh, R. Bloem, R. Ehlers, B. Könighofer, S. Niekum, and U. Topcu, "Safe reinforcement learning via shielding," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 32, no. 1, 2018.

[3] K. P. Wabersich and M. N. Zeilinger, "Linear model predictive safety certification for learning-based control," in *2018 IEEE Conference on Decision and Control*, 2018, pp. 7130–7135.

[4] S. Carr, N. Jansen, S. Junges, and U. Topcu, "Safe reinforcement learning via shielding under partial observability," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 37, no. 12, 2023, pp. 14 748–14 756.

[5] K.-C. Hsu, A. Z. Ren, D. P. Nguyen, A. Majumdar, and J. F. Fisac, "Sim-to-lab-to-real: Safe reinforcement learning with shielding and generalization guarantees," *Artificial Intelligence*, p. 103811, 2023.

[6] K. Stouffer, M. Pease, C. Tang, T. Zimmerman, V. Pillitteri, S. Lightman, A. Hahn, S. Saravia, A. Sherule, and M. Thompson, "Guide to operational technology (OT)

security," National Institute of Standards and Technology, Tech. Rep. Special Publication 800-82 Revision 3, 2023.

[7] MITRE ATT&CK, "Data historian, asset a0006," https://attack.mitre.org/assets/A0006/, 2023, accessed 2026-08-05.

[8] D. T. Phan, R. Grosu, N. Jansen, N. Paoletti, S. A. Smolka, and S. D. Stoller, "Neural simplex architecture," in *NASA Formal Methods*, ser. Lecture Notes in Computer Science, vol. 12229. Springer, 2020, pp. 97–114.

[9] A. Damare, S. Roy, R. Sharma, K. DSouza, S. A. Smolka, and S. D. Stoller, "A barrier certificate-based simplex architecture for systems with approximate and hybrid dynamics," *IEEE Access*, vol. 13, pp. 157 742–157 763, 2025.

[10] W. Zhou, R. Gao, B. Kim, E. Kang, and W. Li, "Runtime-safety-guided policy repair," in *Runtime Verification*, ser. Lecture Notes in Computer Science, vol. 12399. Springer, 2020, pp. 131–150.

[11] P. Lu, M. Cleaveland, O. Sokolsky, I. Lee, and I. Ruchkin, "Repairing learning-enabled controllers while preserving what works," in *2024 ACM/IEEE 15th International Conference on Cyber-Physical Systems*. IEEE, 2024, pp. 1–11.

[12] Y. Chow, O. Nachum, A. Faust, E. Dueñez-Guzman, and M. Ghavamzadeh, "Safe policy learning for continuous control," in *Proceedings of the 2020 Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 155. PMLR, 2021, pp. 801–821. [Online]. Available: https://proceedings.mlr.press/v155/chow21a.html

[13] B. Könighofer, J. Rudolf, A. Palmisano, M. Tappler, and R. Bloem, "Online shielding for reinforcement learning," *Innovations in Systems and Software Engineering*, 2022.

[14] X. Tan, P. Ong, P. Tabuada, and A. D. Ames, "Safety of linear systems under severe sensor attacks," 2024. [Online]. Available: https://arxiv.org/abs/2409.08413

[15] H. Fawzi, P. Tabuada, and S. Diggavi, "Secure estimation and control for cyber-physical systems under adversarial attacks," *IEEE Transactions on Automatic Control*, vol. 59, no. 6, pp. 1454–1467, 2014.

[16] H. Zhang, Z. Li, and A. Clark, "Safe control for nonlinear systems under faults and attacks via control barrier functions," 2022. [Online]. Available: https://arxiv.org/abs/2207.05146

[17] Z. Yuan, A. W. Hall, S. Zhou, L. Brunke, M. Greeff, J. Panerati, and A. P. Schoellig, "Safe-control-gym: A unified benchmark suite for safe learning-based control and reinforcement learning in robotics," *IEEE Robotics and Automation Letters*, vol. 7, no. 4, pp. 11 142–11 149, 2022.

[18] T. P. Kelly and J. A. McDermid, "A systematic approach to safety case maintenance," *Reliability Engineering and System Safety*, vol. 71, no. 3, pp. 271–284, 2001.

[19] E. Denney, G. Pai, and I. Habli, "Dynamic safety cases for through-life safety assurance," in *Proceedings of the 37th International Conference on Software Engineering (ICSE)*, 2015, pp. 587–590.

[20] L. Brunke, M. Greeff, A. W. Hall, Z. Yuan, S. Zhou, J. Panerati, and A. P. Schoellig, "Safe learning in robotics: From learning-based control to safe reinforcement learning," *Annual Review of Control, Robotics, and Autonomous Systems*, vol. 5, pp. 411–444, 2022.

[21] A. Didier, K. P. Wabersich, and M. N. Zeilinger, "Adaptive model predictive safety certification for learning-based control," in *2021 60th IEEE Conference on Decision and Control*, 2021, pp. 809–815.

[22] E. Yu, D. Žikelić, and T. A. Henzinger, "Neural control and certificate repair via runtime monitoring," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 39, no. 25, 2025, pp. 26 409–26 417.

[23] O. Mirzaeedodangeh, E. S. Shekhtman, N. Matni, and L. Lindemann, "Safe planning in interactive environments via iterative policy updates and adversarially robust conformal prediction," in *Proceedings of The 8th Annual Learning for Dynamics and Control Conference*, ser. Proceedings of Machine Learning Research, vol. 331. PMLR, 2026, pp. 678–704. [Online]. Available: https://proceedings.mlr.press/v331/mirzaeedodangeh26a.html

[24] F. Pasqualetti, F. Dörfler, and F. Bullo, "Attack detection and identification in cyber-physical systems," *IEEE Transactions on Automatic Control*, vol. 58, no. 11, pp. 2715–2729, 2013.

[25] L. Brunke, S. Zhou, and A. P. Schoellig, "Barrier bayesian linear regression: Online learning of control barrier conditions for safety-critical control of uncertain systems," in *Proceedings of the 4th Annual Learning for Dynamics and Control Conference*, ser. Proceedings of Machine Learning Research, vol. 168. PMLR, 2022, pp. 881–892. [Online]. Available: https://proceedings.mlr.press/v168/brunke22a.html

[26] A. Capone, R. K. Cosner, A. Ames, and S. Hirche, "Learning safe control via on-the-fly bandit exploration," in *Proceedings of the 42nd International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 267. PMLR, 2025, pp. 6726–6745. [Online]. Available: https://proceedings.mlr.press/v267/capone25a.html

[27] J. Wu, H. Sibai, and Y. Vorobeychik, "Certifying safety in reinforcement learning under adversarial perturbation attacks," 2022. [Online]. Available: https://arxiv.org/abs/2212.14115

[28] L. H. Pham and J. Sun, "Certified continual learning for neural network regression," 2024. [Online]. Available: https://arxiv.org/abs/2407.06697

[29] K.-C. Hsu, D. P. Nguyen, and J. F. Fisac, "ISAACS: Iterative soft adversarial actor-critic for safety," in *Proceedings of The 5th Annual Learning for Dynamics and Control Conference*, ser. Proceedings of Machine Learning Research, vol. 211. PMLR, 2023, pp. 90–103. [Online]. Available: https://proceedings.mlr.press/v211/hsu23a.html

[30] D. P. Nguyen, K.-C. Hsu, W. Yu, J. Tan, and J. F. Fisac, "Gameplay filters: Safe robot walking through adversarial imagination," 2024. [Online]. Available: https://arxiv.org/abs/2405.00846

[31] Y. Ma, X. Zhang, W. Sun, and X. Zhu, "Policy poisoning in batch reinforcement learning and control," in *Advances in Neural Information Processing Systems*, vol. 32, 2019, pp. 14 543–14 553.

[32] X. Zhang, Y. Ma, A. Singla, and X. Zhu, "Adaptive reward-poisoning attacks against reinforcement learning," in *Proceedings of the 37th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 119. PMLR, 2020, pp. 11 225–11 234. [Online]. Available: https://proceedings.mlr.press/v119/zhang20u.html

[33] Y. Sun, D. Huo, and F. Huang, "Vulnerability-aware poisoning mechanism for online RL with unknown dynamics," in *International Conference on Learning Representations*, 2021. [Online]. Available: https://openreview.net/forum?id=9r30XCjf5Dt

[34] Y. Xu, Q. Zeng, and G. Singh, "Efficient reward poisoning attacks on online deep reinforcement learning," *Transactions on Machine Learning Research*, 2025. [Online]. Available: https://openreview.net/forum?id=25G63lDHV2

[35] A. Rakhsha, G. Radanovic, R. Devidze, X. Zhu, and A. Singla, "Policy teaching via environment poisoning: Training-time adversarial attacks against reinforcement learning," in *Proceedings of the 37th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 119. PMLR, 2020, pp. 7974–7984. [Online]. Available: https://proceedings.mlr.press/v119/rakhsha20a.html

[36] T. Everitt, V. Krakovna, L. Orseau, and S. Legg, "Reinforcement learning with a corrupted reward channel," in *Proceedings of the 26th International Joint Conference on Artificial Intelligence (IJCAI)*, 2017, pp. 4705–4713.

[37] S. Jiang, M. Liu, and F. Kong, "Backdoor attacks on safe reinforcement learning-enabled cyber-physical systems," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 43, no. 11, pp. 4093–4104, 2024.

[38] L. Burbano, H. Sasahara, R. Song, Z. B. Celik, and A. A. Cárdenas, "BADControl: Backdoor attacks against control systems," in *Proceedings of the 35th USENIX Security Symposium*, 2026. [Online]. Available: https://www.usenix.org/conference/usenixsecurity26/presentation/burbano

[39] A. Nika, A. Singla, and G. Radanovic, "Online defense strategies for reinforcement learning against adaptive reward poisoning," in *Proceedings of the 26th International Conference on Artificial Intelligence and Statistics*, ser. Proceedings of Machine Learning Research, vol. 206. PMLR, 2023, pp. 335–358. [Online]. Available: https://proceedings.mlr.press/v206/nika23a.html

[40] S. Schaible, "Duality in fractional programming: A unified approach," *Operations Research*, vol. 24, no. 3, pp. 452–461, 1976.

[41] A. Agrawal and S. Boyd, "Disciplined quasiconvex programming," *Optimization Letters*, vol. 14, no. 7, pp. 1643–1657, 2020.

[42] H. H. Bauschke, T. Bendit, and H. Wang, "The homogenization cone: Polar cone and projection," *Set-Valued and Variational Analysis*, vol. 31, no. 3, 2023.

[43] A. Belloni and R. M. Freund, "On the second-order feasibility cone: Primal-dual representation and efficient projection," *SIAM Journal on Optimization*, vol. 19, no. 3, pp. 1073–1092, 2008.

A Deferred Proofs

This appendix collects the proofs deferred from Sections 2 and 4, in the order the results are stated, and adds the action-filtering separation those sections refer to.

Proof of Proposition 1. Each component records a premise under which the acceptance evidence was derived. If an authority transition removes or weakens that premise, the original derivation does not establish the corresponding obligation under C_{deploy}. Re-establishing every changed obligation supplies the missing premises; without them, E alone is insufficient. In the evaluated permanent-release transition, resident authority is removed and a finite check authorizes a longer unmonitored execution, so both authority and horizon change. Under attack, update provenance changes as well. Evidence carried over from protected adaptation also changes subject, from the assured composition to the raw policy. The same argument applies independently to a later handoff from that policy to a baseline controller. □

Proof of Lemma 1. The certified action must satisfy the certificate for every state compatible with the attacked history. These states form $X_{\mathcal{A}}(h)$, and their common certified actions form $K_\Gamma(h)$. Continued certified operation therefore requires $\pi_{\theta_{t+1}}(h) \in K_\Gamma(h)$. An empty kernel requires rejection, freeze, rollback, or trusted information that changes the plausible-state set. □

Proposition 3 (Action filtering is not update certification)**.** *Let $F(v,h)$ be an action filter that returns an applied input in $K_\Gamma(h)$ whenever the kernel is nonempty. Consider a mechanism that first computes $\theta^+ = U(\theta_t, h)$ and then applies*

$$u_t = F(\pi_{\theta^+}(h), h), \tag{10}$$

The mechanism accepts the update without constraining the raw updated policy π_{θ^+}. If $\pi_{\theta^+}(h) \notin K_\Gamma(h)$, the applied input can be certified while the updated learner remains uncertified. Action-only filtering therefore does not imply Lemma 1.

Proof of Proposition 3. The filter property gives $u_t \in K_\Gamma(h)$, so the current actuator input can satisfy the certificate. Lemma 1 instead constrains the updated controller through $\pi_{\theta^+}(h) \in K_\Gamma(h)$. When this membership fails, the certificate covers only the filtered actuator channel. Accepting the update under that certificate creates a stale certified policy. □

Proof of Theorem 1. Applying Lemma 1 pointwise to every $h' \in \mathcal{H}_{\mathcal{A}}(h, \theta^+)$, which the soundness definition of Section 2 quantifies over, gives $\pi_{\theta^+}(h') \in K_\Gamma(h')$ for each such history. This is the definition of $\theta^+ \in \Theta_\Gamma(h)$. A failed membership leaves the updated action map uncertified at that history. An empty kernel additionally requires a changed information set or rejection of certified operation. □

Proof of Proposition 2. For fixed z, membership $\pi_\theta(z) \in [\ell_z, u_z]$ is equivalent to the two inequalities of (9). Enforcing them for every $z \in \mathcal{Z}$ implements the lifecycle gate on the abstraction. Parameter box constraints preserve convexity. A feasible or projected candidate satisfies every certified interval kernel in the abstraction. □

Proof of Proposition 5. The standardized advantage vector is $y/\sqrt{T^{-1}\|y\|_2^2} = \sqrt{T}\, y/\|y\|_2$. Substitution into the usual mean score-function gradient $T^{-1}J^\top(\sqrt{T}\, y/\|y\|_2)$ gives (24). Equation (23) is affine in the reward-box variable, but division by its Euclidean norm is not; hence the normalized image is generally non-affine. □

B Continuous Lift, Kernel Instantiation, and a Minimal Counterexample

B.1 A Margin-Robust Finite-Abstraction Lift

Proposition 2 is exact on its listed grid. Lifting that result to continuous reachable histories requires coverage and regularity assumptions. Let $z = \psi(h')$ be a finite-dimensional representation on which the updated policy depends. Suppose the fixed-controller certificate can be written as finitely many obligations

$$c_j(x,u,w,x^+) \le 0, \qquad j = 1,\dots,m, \tag{11}$$

where $x^+ = f(x,u,w)$. Invariant, discrete-time barrier, and bounded-horizon reachability checks can be written in this form after augmenting the state with time or certificate memory.

Theorem 2 (Margin-robust finite-abstraction sufficiency)**.** *Fix a candidate snapshot π_θ and a future-history horizon. Let $\{(z_i, x_{iq}, w_{ir})\}$ be a finite set such that, for every reachable attacked history h', every $x \in X_{\mathcal{A}}(h')$, and every $w \in \mathcal{W}$, some sampled triple satisfies*

$$\|z - z_i\| \le \delta_z, \quad \|x - x_{iq}\| \le \delta_x, \quad \|w - w_{ir}\| \le \delta_w, \tag{12}$$

where $z = \psi(h')$. Assume:

1. *$\pi_\theta(z) \in \mathcal{U}$ and $\|\pi_\theta(z) - \pi_\theta(z')\| \le L_\pi \|z - z'\|$;*
2. *the certificate model $\widehat{f}$ has uniform error $\|f(x,u,w) - \widehat{f}(x,u,w)\| \le \varepsilon_f$;*
3. *$\widehat{f}$ is separately Lipschitz with constants L_f^x, L_f^u, L_f^w; and*
4. *each c_j is separately Lipschitz in (x,u,w,x^+) with constants $L_j^x, L_j^u, L_j^w, L_j^+$.*

Define

$$\begin{aligned}\eta_j = {} & (L_j^x + L_j^+ L_f^x)\delta_x \\ & + (L_j^u + L_j^+ L_f^u) L_\pi \delta_z \\ & + (L_j^w + L_j^+ L_f^w)\delta_w + L_j^+ \varepsilon_f.\end{aligned} \tag{13}$$

If every sampled triple satisfies

$$c_j\Big(x_{iq}, \pi_\theta(z_i), w_{ir}, \widehat{f}(x_{iq}, \pi_\theta(z_i), w_{ir})\Big) \le -\eta_j \tag{14}$$

for all j, then $\pi_\theta(\psi(h')) \in K_\Gamma(h')$ for every reachable attacked history h' in the horizon. Hence the snapshot satisfies the continuous future-history lifecycle condition of Theorem 1.

Proof. Fix h', $x \in X_{\mathcal{A}}(h')$, and $w \in \mathcal{W}$, and select a sampled triple using (12). Write $u = \pi_\theta(z)$ and $u_i = \pi_\theta(z_i)$. Policy regularity gives $\|u - u_i\| \le L_\pi \delta_z$. The model-error and Lipschitz assumptions give

$$\begin{aligned}\|f(x,u,w) - \widehat{f}(x_{iq}, u_i, w_{ir})\| \le \varepsilon_f + L_f^x \delta_x \\ + L_f^u L_\pi \delta_z + L_f^w \delta_w.\end{aligned} \tag{15}$$

Applying the four Lipschitz bounds of c_j and collecting terms yields

$$\begin{aligned}c_j(x,u,w,f(x,u,w)) &\le c_j(x_{iq}, u_i, w_{ir}, \\ &\quad \widehat{f}(x_{iq}, u_i, w_{ir})) + \eta_j \\ &\le 0.\end{aligned} \tag{16}$$

The argument holds for every plausible state, disturbance, certificate obligation, and reachable attacked history, which is precisely membership in the corresponding kernels over the stated horizon. □

Theorem 2 converts a justified joint cover into a verification result. Establishing that cover remains a separate obligation; a finite rollout sample is insufficient. The coverage audit in Appendix H provides empirical evidence about this premise without constituting a cover proof.

Proposition 4 (Plausible-set and freeze-frontier monotonicity)**.** *Fix a history domain $\mathcal{H}$, dynamics, certificate obligations, and policy class. If two information contracts satisfy $X_1(h) \subseteq X_2(h)$ for every $h \in \mathcal{H}$, let $K_{\Gamma,k}$ be the kernel induced by X_k and let $\Theta_{\Gamma,k} = \{\theta : \pi_\theta(h) \in K_{\Gamma,k}(h),\ \forall h \in \mathcal{H}\}$. Then*

$$K_{\Gamma,2}(h) \subseteq K_{\Gamma,1}(h) \quad \text{and} \quad \Theta_{\Gamma,2} \subseteq \Theta_{\Gamma,1}. \tag{17}$$

Consequently, for a nested ambiguity family $X_{\rho_1}(h) \subseteq X_{\rho_2}(h)$ whenever $\rho_1 \le \rho_2$, certified update feasibility is nonincreasing in ρ: once the certified parameter set is empty, it remains empty at larger ambiguity. Conversely, a trusted anchor that pointwise shrinks the plausible-state sets can only weakly enlarge the kernels and certified parameter set.

Proof. An action certified for every state in the larger set $X_2(h)$ is certified for every state in its subset $X_1(h)$, proving kernel inclusion. Requiring a policy action to lie in the smaller kernel $K_{\Gamma,2}(h)$ for every $h \in \mathcal{H}$ is at least as restrictive as requiring membership in $K_{\Gamma,1}(h)$, proving parameter-set inclusion. The frontier and anchor statements follow by applying these inclusions to nested families. □

The monotonicity comparison holds on a fixed history domain and certificate contract. If an anchor changes which histories are reachable, or if two sampled plausible sets are not nested, neither conclusion follows automatically. The P3 anchor audit therefore constructs nested samples explicitly.

The three results have distinct scopes. The future-history theorem states a necessary certificate condition. Proposition 2 is exact on its finite abstraction. Theorem 2 supplies a sufficient lift under its cover, regularity, and margin assumptions.

B.2 Instantiating the Kernel

For an invariant certificate $I \subseteq S$, the kernel is

$$\begin{aligned}K_I(h) = \{u \in \mathcal{U} : \forall x \in X_{\mathcal{A}}(h), \forall w \in \mathcal{W}, \\ f(x,u,w) \in I\}.\end{aligned} \tag{18}$$

For a barrier certificate $B(x) \ge 0$ with safe set $C = \{x : B(x) \ge 0\}$, the corresponding kernel is

$$\begin{aligned}K_B(h) = \{u \in \mathcal{U} : \forall x \in X_{\mathcal{A}}(h), \forall w \in \mathcal{W}, \\ B(f(x,u,w)) \ge 0\}.\end{aligned} \tag{19}$$

A reachability certificate starts from $X_{\mathcal{A}}(h)$ rather than a nominal state estimate. A runtime shield must select an action in $K_\Gamma(h)$. An empty kernel requires rejection, fail-safe operation, or a trusted state anchor.

B.3 A Minimal Counterexample

Consider

$$x_{t+1} = \lambda x_t + u_t, \qquad |u_t| \le u_{\max}, \qquad S = [-1,1]. \tag{20}$$

Suppose FDI makes the same history compatible with $x_t = r$ and $x_t = -r$, where $r \in (0,1]$. The one-step safe action set for state x is

$$\mathcal{U}_{\text{safe}}(x) = [-u_{\max}, u_{\max}] \cap [-1 - \lambda x, 1 - \lambda x]. \tag{21}$$

For $x = r$ and $x = -r$, these sets are individually nonempty but disjoint whenever

$$1 < \lambda r < 1 + u_{\max}. \tag{22}$$

The strict upper bound makes each known state individually controllable. FDI creates indistinguishability, so no single history-dependent action certifies both states.

This example distinguishes fixed filtering from certified learning. A fixed severe-attack filter can reject when the common kernel is empty. A certified online learner must also prevent updates under a certificate with no feasible kernel.

C Exact Positive Support Under Gradient Clipping

This appendix states the one-batch reward-influence results summarized in Section 4.2. Theorem 3 is proved here; Proposition 5 is proved with the other deferred results in Appendix A.

C.1 Normalized One-Batch Proposal Set

This subsection connects the reward-record attacker to the release halfspaces of Proposition 2. For a fixed on-policy batch of length T, let $J \in \mathbb{R}^{T\times d}$ contain the policy-score row vectors, let $r \in \mathbb{R}^T$ be the logged rewards, let $H_{ij} = \mathbf{1}[j \geq i]$ be the reward-to-go operator, and let $C = I - T^{-1}\mathbf{1}\mathbf{1}^\top$ be the centering matrix. An attacker choosing $\delta \in [-\varepsilon, \varepsilon]^T$ produces the centered return vector

$$y(\delta) = CH(r+\delta). \tag{23}$$

The fixed-batch assumption is appropriate for a reward-record-only attacker: changing a reward after collection does not change the states, actions, or score rows already in the batch.

Proposition 5 (Standardized reward-influence set)**.** *Suppose the learner standardizes the centered reward-to-go advantages with population standard deviation whenever $\|y(\delta)\|_2 > 0$. Before optimizer clipping, a gradient-ascent proposal of size α is*

$$\widetilde{\theta}^+(\delta) = \theta + \frac{\alpha}{\sqrt{T}} J^\top \frac{y(\delta)}{\|y(\delta)\|_2}. \tag{24}$$

Consequently, the raw one-batch proposal set is the image of the centered reward-to-go zonotope under normalization and the policy-score map; in general, it is not a parameter-space zonotope.

Normalization is why a bounded reward box does not induce a bounded parameter-space zonotope, and it is also what makes the influence question tractable. The support characterization below applies to one fixed batch; later batches change J, r, and the visited states. The implementation skips standardization below a numerical tolerance, so each reported cone witness must remain above that threshold.

C.2 Exact Positive Support

Theorem 3 (Positive support under global gradient clipping)**.** *Let $B = J^\top/\sqrt{T}$, let the learner cap the global gradient norm at $G > 0$, and assume coordinatewise parameter clipping is inactive over the reward box. The post-cap update before parameter clipping is*

$$\theta_G^+(y) = \theta + \frac{\alpha B y}{\max\{\|y\|_2, \|By\|_2/G\}}. \tag{25}$$

For a release halfspace $a^\top P\theta \leq b$, define $q = \alpha B^\top P^\top a$, $y_0 = CHr$, and $M = CH$. For any fixed $t > 0$, a reward perturbation whose gate-coordinate increment is at least t exists if and only if the following second-order-cone problem is feasible:

$$\begin{aligned} &\lambda \geq 0, \qquad -\varepsilon\lambda\mathbf{1} \leq v \leq \varepsilon\lambda\mathbf{1}, \\ &\bar{y} = \lambda y_0 + Mv, \qquad q^\top \bar{y} = 1, \\ &t\|\bar{y}\|_2 \leq 1, \qquad (t/G)\|B\bar{y}\|_2 \leq 1. \end{aligned} \tag{26}$$

Feasibility is monotone in t, so bisection on $[0, \min\{\|q\|_2, \alpha G\|P^\top a\|_2\}]$ computes the exact positive support

$$\max\left\{0, \sup_{\|\delta\|_\infty \leq \varepsilon} \frac{q^\top y(\delta)}{\max\{\|y(\delta)\|_2, \|By(\delta)\|_2/G\}}\right\}. \tag{27}$$

The construction remains valid when the centered-return zonotope contains the origin; if $G = \infty$, the second norm constraint is omitted. If this support does not exceed the current halfspace margin, no admissible reward edit can cross that halfspace in the batch.

Proof. Global norm clipping gives the stated maximum in the denominator. The ratio is positively homogeneous, so optimization over the zonotope $Z = \{y_0 + M\delta : \|\delta\|_\infty \leq \varepsilon\}$ is equivalent to optimization over its conic hull. The first line of (26) is the perspective representation of that hull: $v = \lambda\delta$. This is the standard homogenization used in fractional and conic optimization [40, 42].

If a positive ratio reaches t, rescale its conic representative until $q^\top \bar{y} = 1$; its two denominator branches are then exactly the last line of (26). Conversely, any feasible $\bar{y}$ is nonzero because of the unit-numerator equality and recovers a reward-box ray whose ratio reaches t. Thus the origin cannot create the degenerate feasibility present in a direct zonotope formulation. Each fixed-t problem is a second-order feasibility-cone problem [43]; monotonicity permits quasi-convex bisection [41]. Cauchy–Schwarz supplies both stated upper bounds. The margin result follows by adding the maximum increment to the current gate coordinate. □

D Reward-Influence Implementation Audit

This appendix expands the batch-level audit summarized in Section 5.

We first validated the reward-influence analysis on an unclipped cartpole batch from the learner indexed by seed 2040. Reconstructing the implemented gradient from Equation (24) has error 2.78×10^{-17}. The clean gradient norm is 0.196789, neither gradient nor parameter clipping is active, and the minimum centered-return norm over the reward box is 7.993864. Across 22 release-gate halfspaces, the largest discrepancy between an SOCP witness and its computed support is 1.07×10^{-9}. The minimum robust margin was 4.598004, so no halfspace could be crossed in this batch. The successful physical attack instead used the locked bounded tanh target-shaping rule and accumulated influence across changing batches. It was not generated by a one-step SOCP.

Two formulations of the same support question differ in what they can admit. The direct zonotope formulation excludes any batch whose reward box contains a normalization singularity, and any batch under active global gradient clipping; on the 36 poisoned V3 batches it admits none. The

Table 6: Commit availability and offline computation on the independent 144-state candidate pool. Common admission requires both freeze and the committed snapshot to pass the 100-step, 0.003-guard certificate.

Seed	Common/144	Commit frac.	Offline (s)
2040	143/144 (99.3%)	0.55	8.49
2041	144/144 (100.0%)	0.40	9.32
2042	130/144 (90.3%)	0.70	8.50

homogenized second-order-cone formulation of Theorem 3 yields verified support witnesses on 22 of the 36, including 18 batches with a singular reward box and 20 that the clipping exclusion would have rejected. Every actual update is below its computed support, all witnesses respect the reward budget, final parameter reconstruction error is zero, and maximum witness/support error is 3.67×10^{-5}. Ten batches cannot globally exclude coordinatewise parameter clipping (nine activate it on the observed edit), and four cone-optimal directions have no recovered witness above the learner's normalization tolerance; they remain outside the exact implementation audit. The clipped quadrotor trajectories have not received this separate parameter-clipping audit and remain empirical.

Exact batch support did not explain the successful multi-batch attack. The locked tanh update advances in the target direction on only 6/20 eligible batches with positive oracle increment, and its preregistered same-trajectory advantage reaches 22/36 rather than the required 27/36. A diagnostic rerun on the development realization under the homogenized formulation emitted nonzero edits on 9 of its 12 batches, where the direct formulation had emitted none, and its accepted raw snapshot still caused 0/24 release failures. The exact-support result therefore serves as a batch audit. The physical experiment evaluates the fixed target and bounded attacker without establishing joint gate optimality of the tanh rule.

E Additional Boundaries and Validity Checks

This appendix expands the boundary audits summarized in Section 5.

Trusted-state-anchor infrastructure. A trusted state anchor shrinks certificate uncertainty but does not sanitize the reward record. Varying the anchor period $P \in \{1,3,6,12\}$ (12, 4, 2, or 1 calls over 12 updates), every condition retains all 16 certificate centers, has 0/24 violations on new paired blocks, and improves no reward over freeze; retained fractions stay at 0.55/0.40/0.70 for $P \leq 6$ and become 0.50/0.40/0.65 at $P = 12$. Hardware latency, energy, authentication, and monetary cost are not measured.

Official PPO. The official pretrained PPO policy is safe on 72/72 held-out rollouts, while a separately constructed malicious target is unsafe on 72/72. Under the locked $L_\infty \leq 0.5$ reward budget, the 16-batch checkpoint has 0/24 clean and 1/24 poisoned failures; at 24 batches both have 0/24. The pre-specified physical and two-checkpoint margin conditions failed, so the planned multi-run extension was not executed. The audit showed checkpoint-level update redirection without a stable deep-PPO unsafe-policy effect.

Table 7: Frozen reward-integrity audit. Rates are batch-level true- and false-positive rates (TPR/FPR) across seven training runs. Each check requires the stated defender assumption.

Check	Required trusted information	TPR/FPR
Reward recomputation	Authenticated transition and deterministic reward	1.000/0
Known-sign check	Task guarantee that reward is nonpositive	0.905/0
Scalar envelope	Clean development range transfers	0.917/0.036
Batch-mean envelope	Clean batch mean transfers	0.012/0.036

Benign adaptation utility. In the locked benign-utility audit under a 0.003 N differential actuator bias, freeze, clean adaptation, and LifecycleGate all have 0/12 violations and mean rewards -0.03617, -0.04714, and -0.08097. LifecycleGate accepts a nonzero update in 11/12 batches (mean fraction 0.8208) but improves no paired rollout over freeze. This met the preregistered stopping criterion, so the planned three-run extension was not executed. Parameter motion and certificate acceptance did not establish useful adaptation in this setting.

Measurement. The CasADi gates use fixed finite state sets; physical outcomes use independent PyBullet rollouts. They are sampled finite-horizon certificates, not continuous-state invariant-set proofs. For the benign shift, maximum action implementation error is 2.61×10^{-11} N. A corrected calibration metric left run indices, states, bias grid, and threshold unchanged. Realized state violations and pre-clip saturation are reported separately.

Integrity controls and negative variants. Four frozen checks were evaluated on 84 clean and 84 poisoned batches from seven training runs. Trusted recomputation blocks every evaluated edit when reward is a deterministic function of the authenticated transition. The known-sign and scalar-envelope checks detected above 90% of the original sign-violating rule, whereas a batch-mean envelope did not. Temporal and moment-preserving variants respected their stated invariants and moved parameters toward the target but each caused 0/24 raw-release failures; those experiments bound two hand-designed variants, not the channel. Placing the known-sign check in the update loop reduced the original rule from 39/126 raw-release failures to 0/126, but Section 5.4 shows that a constrained attacker retains harm. Authentication and trusted reward recomputation therefore provide upstream controls under different assumptions. That producer remains within the trusted computing base of the evaluated threat model.

F Additional Third-Party Batteries

This appendix reports the learner-family, attack-budget, and second-environment batteries summarized in Section 6.

F.1 Learner Boundary

The same contrast attack (δ=2) was evaluated on the third-party PPO pipeline with action masking and sudden retirement. It raised post-removal violations in one of three paired runs, from 1178 to 2439 (McNemar exact $p = 2.6 \times 10^{-149}$). The first-violation episode changed from 6 to 0. The other two runs were unchanged ($p = 1.00$ and 0.96). In a separate ten-run shield-on-only isolation battery, the run indexed by configured seed 101 was positive ($0.219 \to 0.754$ at retirement; McNemar exact $p = 3.7 \times 10^{-113}$), while the other nine entered a high-violation ceiling regime (clean 0.728–0.760) and were either protective or unchanged. Fig. 4 reports the full battery. A ten-run full-phase PPO battery likewise placed every clean policy near the retirement ceiling (0.73–0.77) and yielded no positive outcome on the primary retirement metric. Its final metric captured a different effect: all ten poisoned policies remained unsafe (0.49–0.77), whereas nine clean policies recovered to approximately 0.22 under continued learning.

A ten-run shield-on-only battery on off-policy SAC provided a complementary boundary. Three runs were positive ($0.207 \to 0.561$, $0.218 \to 0.513$, and $0.479 \to 0.954$; paired McNemar exact $p < 10^{-40}$ for each), three were protective, and four were unchanged. All ten clean SAC policies had low retirement disagreement rather than occupying the ceiling regime. Four extension runs generated on a second host were non-positive. The fixed attack therefore affected individual PPO and SAC training realizations without producing reliable success within either family.

F.2 Attack Budget and State Dependence

On the full-phase REINFORCE protocol, the effect varied with attack shape, budget, and training realization. In the realization indexed by configured seed 2, δ=2 raised post-removal violations from 1127 to 3762 (3.3×; McNemar exact p below numerical reporting precision). Increasing the budget to δ=10 produced 3766 violations. In the realization indexed by seed 3, δ=2 was unchanged (2482 vs. 2471; $p = 0.84$), whereas δ=10 raised violations to 3700 ($p = 8.4 \times 10^{-138}$). The realization indexed by seed 1 began near the unsafe ceiling (75.7%) and showed no added damage at either budget. Shape controls were also state-dependent. The bias variant was protective in the ceiling realization. The risk variant was protective in two realizations and raised violations from 1127 to 2457 in the realization indexed by seed 2.

An independent Linux-host batch with identical dependency versions contained one low-clean-violation realization and two ceiling realizations. The former showed the contrast effect at retirement ($0.232 \to 0.501$) and after removal (1157 $\to$ 2451; $p = 4.6 \times 10^{-159}$). The latter two were unchanged at δ=2 and protective at δ=10. The run index of the low-clean-violation regime differed across hosts. Repeating the clean configuration also entered a different trajectory mode (0.776 rather than 0.232 at retirement). Dose comparisons are therefore confined to the locked local batch. The independent batch shows that attackability follows the realized retirement state more closely than the nominal seed identifier. Fig. 5 summarizes both panels.

F.3 Environment Boundary

On a second gridworld of the same third-party suite (surveillance, N=6, R=3), a clean reproduction again satisfied the authors' lifecycle ordering: shield retained $0/0$, smooth 0.129, sudden 0.246, and no-shield 0.252 (retained $<$ smooth $\ll$ sudden $\leq$ no-shield). The fixed contrast attack at δ=2 raised post-removal violations in one of three independent paired runs ($0.000 \to 0.272$, McNemar exact $p = 2.6 \times 10^{-82}$) and was protective in the other two ($0.267 \to 0.000$, $p = 8.4 \times 10^{-81}$; $0.135 \to 0.038$, $p = 1.3 \times 10^{-15}$). The lifecycle ordering extended to the second environment, whereas the exploit effect remained environment- and realization-dependent.

F.4 Opportunity-Region Cross-Tabulation

Fig. 7 plots the cross-tabulation reported in Section 6. Fig. 6 instead uses the obstacle-only marker records that contain disagreement instrumentation. Every retained positive outcome has clean $D_{\text{ret}} \leq 0.047$, and no high-band run is positive. The configuration-level accounting and the clean-versus-clean null of Section 6 are produced by `analyze_two_sided_and_null.py` in the artifact, which writes both row-level tables.

F.5 Clean-Versus-Clean Null

A clean run carries no poisoning under either scope label, so two independently trained clean records of one configuration under different scope labels form a clean replicate pair. The obstacle-only aggregate table contains 21 such pairs. Eleven share a code version: one exceeds the 0.15 decision threshold, at $|\Delta| = 0.540$, and the median absolute difference over the eleven is 0.007. Ten cross code versions, of which three exceed the threshold, at 0.548, 0.503, and 0.498. The differences are bimodal: every pair either lands within 0.03 or changes by nearly half the outcome range.

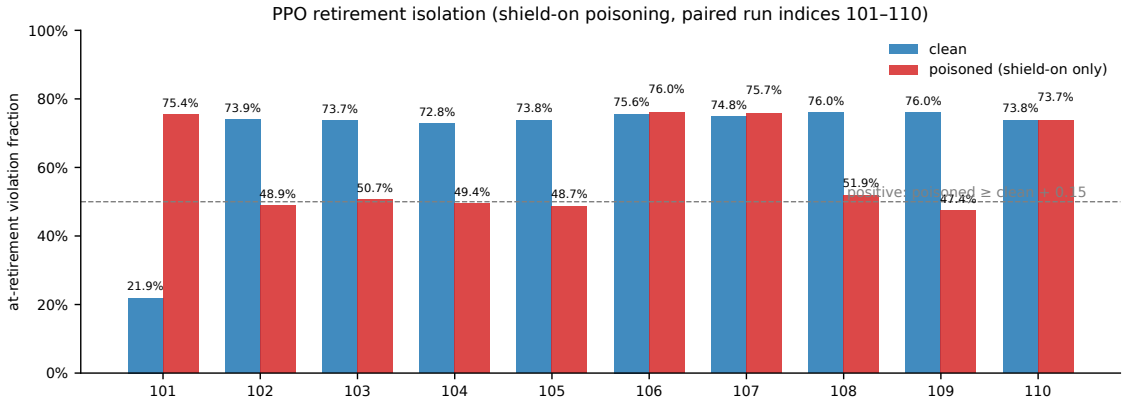

Figure 4: PPO retirement-boundary isolation on the third-party lifecycle (shield-on-only contrast poisoning, δ=2, paired runs indexed by run indices 101–110, sudden retirement). At-retirement violation fraction of the raw policy for clean and poisoned runs. The run indexed by 101 is positive (0.219 → 0.754; McNemar exact $p = 3.7 \times 10^{-113}$); the other nine runs land in the unsafe ceiling regime (clean at-retirement 0.728–0.760) with no paired headroom, six protective by the paired test and three unchanged. The dashed line marks the locked decision boundary (poisoned at least 0.15 above clean with paired McNemar $p < 0.01$).

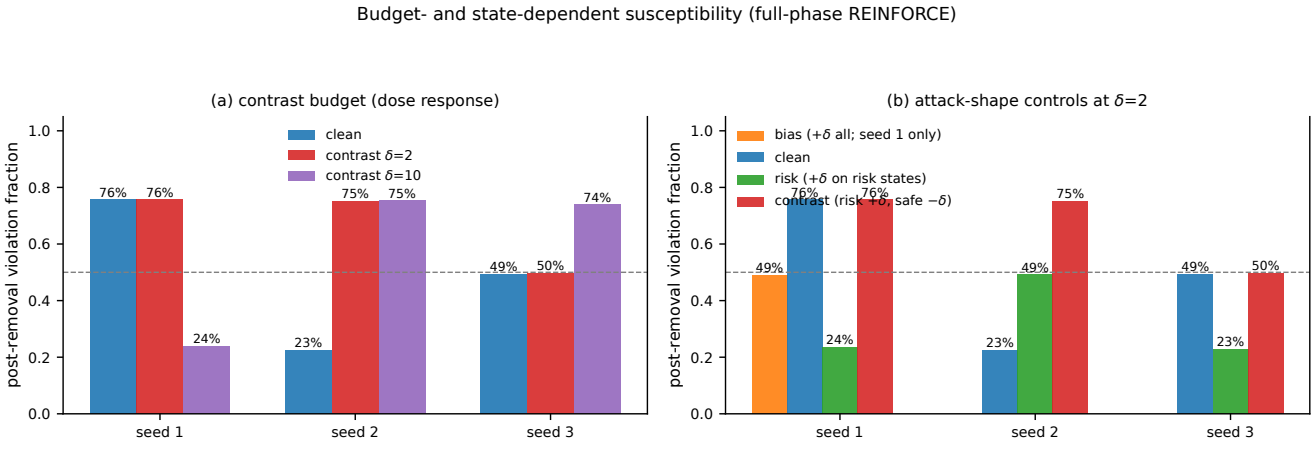

Figure 5: Budget- and state-dependent effects on the full-phase REINFORCE protocol (paired within one locked code version). (a) Contrast attack dose response; (b) attack-shape negative controls at δ=2. The run-index-2 realization saturates at δ=2; the run-index-3 realization requires δ=10; the ceiling realization shows no additional damage at either budget, and bias/risk controls are protective or strictly weaker.

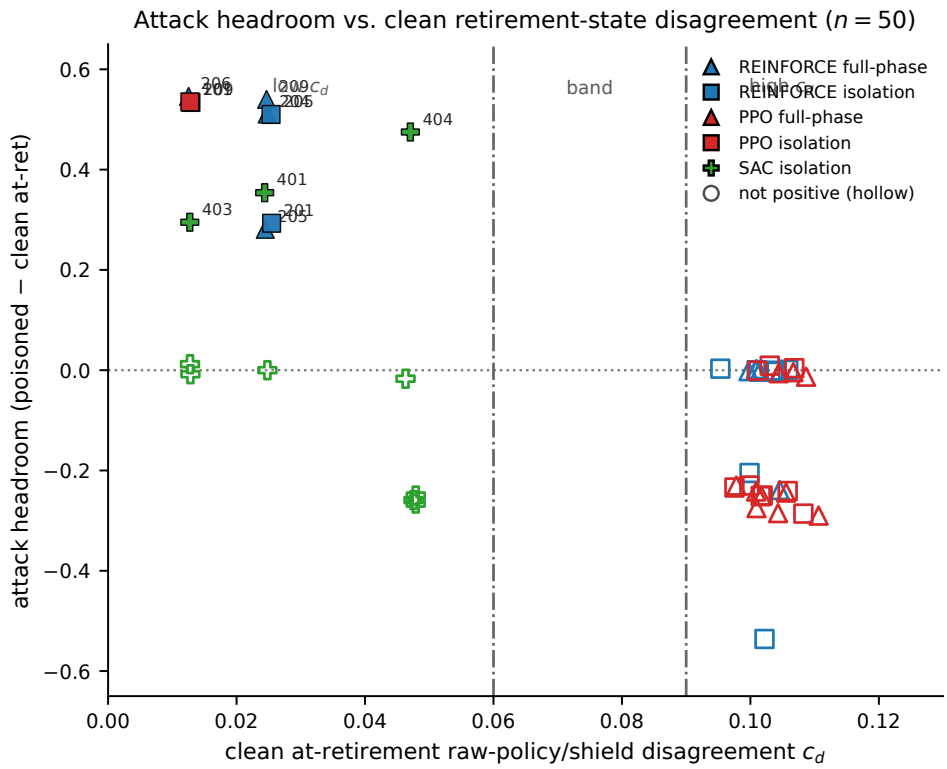

Figure 6: Clean at-retirement raw-policy/shield disagreement marks an opportunity region across the obstacle corpus (both scopes, all three learner families). Each point is one paired clean/poisoned run; filled markers meet the positive-outcome rule, and hollow markers do not. Filled markers all have clean disagreement ≤ 0.047; the dashed lines mark the fixed low/high disagreement bands (< 0.06 and ≥ 0.09). No high-disagreement run is positive, while the low-disagreement region contains both positive and non-positive outcomes. Exact corpus counts and a discrimination statistic are not reported because two historical deduplication decisions are unrecoverable.

F.6 Reproducible Exclusion Audit and Band Chronology

Before historical membership decisions, the executable marker audit emits 58 configuration rows: 15 positive and 43 non-positive. Two REINFORCE membership decisions used for the retained marker figure lack an executable deduplication rule, so those raw totals are not treated as the marker's analysis denominator. Membership-invariant quantities are 32 high-band rows with no positive outcome, a maximum clean disagreement of 0.047 among retained positive rows, and non-positive rows extending to 0.111. All 20 PPO rows satisfy the exclusion; all ten SAC rows are low-band, and only three are positive. A corpus-level discrimination statistic and an exact low-band success fraction therefore remain unreported.

The protocol chronology bounds the statistical interpretation, and the three analysis thresholds have different chronologies. The positive-pair rule (poisoned at-retirement at least 0.15 above clean, paired McNemar $p < 0.01$) was written into the protocol on 2026-08-16, after the first three REINFORCE isolation pairs had been observed and before every later battery, so those three pairs are labelled by a rule that postdates them while later runs were labelled prospectively. The clean-violation 0.5 cut was likewise read off the early rows and then pre-specified for the ten-run batteries. The 0.06/0.09 bands were fixed after analysis of the then-existing obstacle rows and before four SAC extension pairs with run indices 407–410. All four extensions were low-band and non-positive. They strengthened the non-sufficiency result without prospectively testing high-disagreement exclusion. The marker is also obstacle-scoped: two protective runs in the second environment had clean disagreement 0.476 and 0.157 despite clean violation below 0.5.

G LifecycleGate Semantic Benchmarks

This appendix contains the low-dimensional semantic benchmarks that distinguish current-action filtering from update admission. They instantiate the certificate semantics of Section 4 rather than the authority-transition attack evaluated in Section 5.

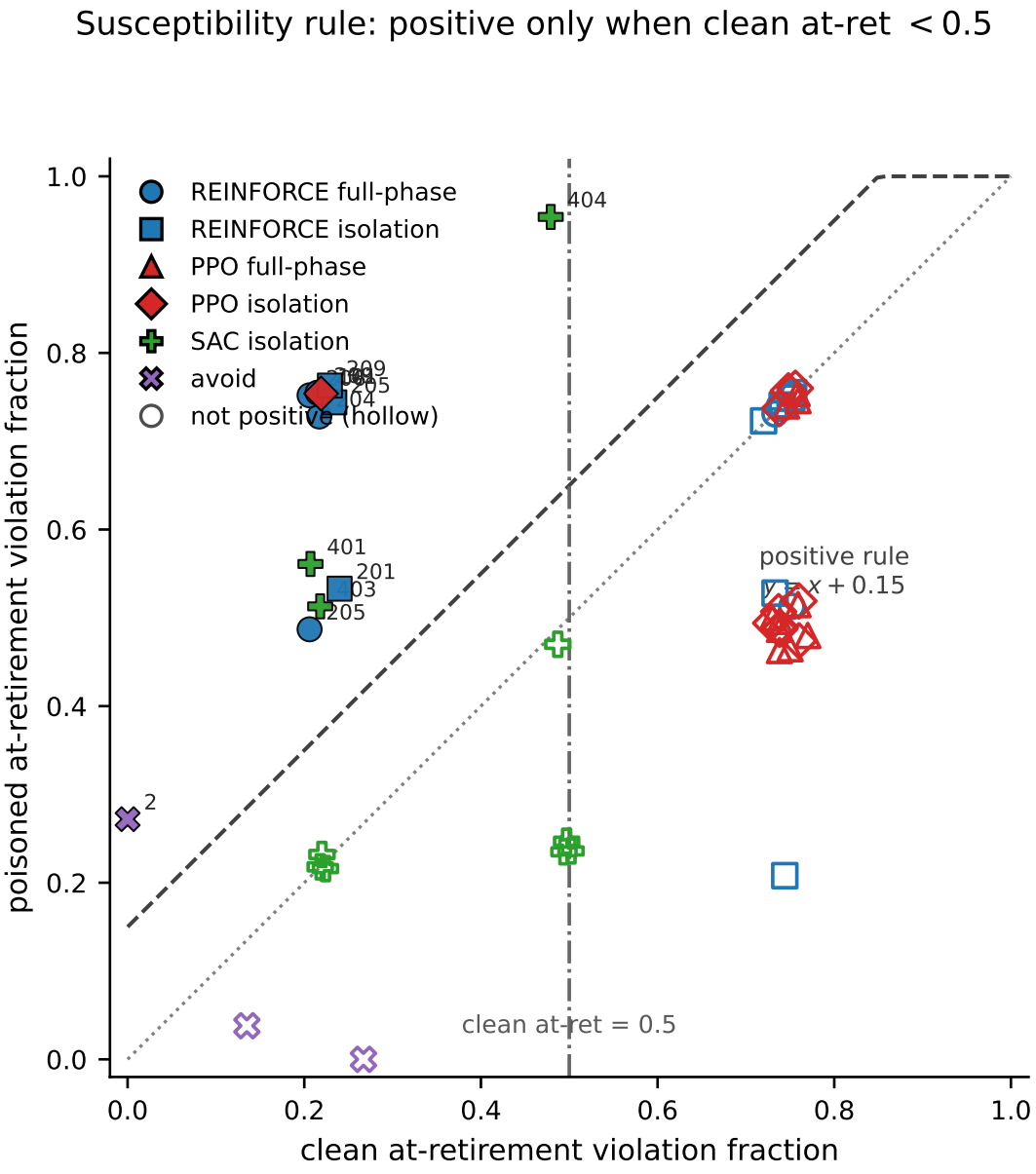

Figure 7: Two-by-two opportunity-region cross-tabulation across all locked paired runs (both scopes, all three learner families, both domains). Each point is one paired clean/poisoned run; filled markers meet the positive rule (poisoned at-retirement at least 0.15 above clean at retirement with paired McNemar exact $p < 0.01$), hollow markers do not. The dashed rule line is $y = x + 0.15$ and the vertical dashed line marks the clean at-retirement 0.5 boundary: positive outcomes concentrate below 0.5, and no ceiling run is positive.

Table 8: Certified kernel for the one-dimensional ambiguity set.

ρ	$\lambda\rho$	$K(\rho)$	Gate
0.600	0.720	$[-0.200, 0.200]$	allow
0.800	0.960	$[-0.040, 0.040]$	constrain
0.833	1.000	$\{0\}$	boundary
0.900	1.080	$\emptyset$	freeze/reject
1.000	1.200	$\emptyset$	freeze/reject

G.1 Setup

We instantiated the linear system of Appendix B with $\lambda = 1.2$, $u_{\max} = 0.2$, and $S = [-1, 1]$. FDI induced the ambiguity set $X_{\mathcal{A}}(h) = [-\rho, \rho]$. The common certified kernel for invariant certificate $I = S$ is

$$K(\rho) = [-u_{\max}, u_{\max}] \cap [-1 + \lambda\rho, 1 - \lambda\rho]. \tag{28}$$

The kernel is nonempty exactly when $\lambda\rho \leq 1$.

G.2 Learning-Freeze Frontier

The frontier occurs at $\rho = 1/\lambda = 0.833$. Table 8 shows the exact kernel values for representative ambiguity radii.

At $\rho = 0.9$, states $+0.9$ and -0.9 are individually controllable when their sign is trusted. Their safe intervals are $[-0.2, -0.08]$ and $[0.08, 0.2]$, respectively. The empty intersection prevents a common history-dependent action from being certified.

G.3 Trusted Anchors and Learning-Aware FDI

A trusted state anchor can restore learning by shrinking the plausible-state set. At $\rho = 0.9$, a trusted sign anchor reduces the set to $[0, 0.9]$ or $[-0.9, 0]$. The corresponding kernels, $[-0.2, -0.08]$ and $[0.08, 0.2]$, are nonempty.

Learning-aware FDI creates the complementary failure mode. For a candidate action $u = 0.1$, the gate allows the update at $\rho = 0.6$, projects it to 0.04 at $\rho = 0.8$, and rejects it at $\rho = 0.9$. The attacked history therefore determines whether the same candidate is allowed, constrained, or rejected.

G.4 Lifecycle Benchmark Results

We compared five mechanisms over five update steps: an ungated learner, a nominal action filter, a robust action filter, always-freeze, and LifecycleGate with projection. The one-dimensional benchmark used the system above. The linear tank benchmark used the level-deviation model

$$x_{t+1} = Ax_t + Bu_t, \quad A = \begin{bmatrix} 1.08 & 0.04 \\ 0.02 & 1.03 \end{bmatrix}, \quad B = \begin{bmatrix} 1 \\ 0.1 \end{bmatrix}, \tag{29}$$

with $|u_t| \leq 0.15$ and $x_t \in [-1, 1]^2$. The nonlinear tank benchmark uses square-root inter-tank and drain flows,

$$\begin{aligned} h_{1,t+1} &= h_{1,t} + \Delta t(q_0 + u_t - q_{12}(h_t) - d_1(h_{1,t})), \\ h_{2,t+1} &= h_{2,t} + \Delta t(q_{12}(h_t) - d_2(h_{2,t})), \end{aligned} \tag{30}$$

where q_{12} and d_i are square-root flow terms. Its interval observer returned a box around the nominal attacked history. The one-step kernel was computed over a grid within that box. In all benchmarks, learning-aware FDI pushed the learner toward a larger positive action parameter.

Fig. 8 separates current-action filtering from update certification. At $\rho = 0.8$, the nominal filter had unsafe certified rate 1.0. The robust action filter had stale-policy rate 1.0 because the learner parameter left $K(\rho)$ after action projection. LifecycleGate had zero unsafe or stale operation and retained one of five updates. At $\rho = 0.9$, the kernel was empty and LifecycleGate froze learning.

The linear tank benchmark reproduced the same pattern. At $\rho = 0.8$, nominal filtering had unsafe certified rate 0.8, and robust action filtering had stale-policy rate 0.8. LifecycleGate kept both rates at zero and allowed two of five updates. It allowed one constrained update at $\rho = 0.85$ and froze at the empty-kernel point $\rho = 0.92$.

The nonlinear tank benchmark preserved the separation with kernels computed from sampled nonlinear dynamics. At $\rho = 0.2$, nominal filtering was unsafe in four of five steps, and robust filtering was stale in four. LifecycleGate kept both rates at zero while allowing two constrained updates. At $\rho = 0.45$, the interval-observer box admitted no common safe action and LifecycleGate froze the update.

G.5 Parameter-Level Update Gate

We also instantiated the gate at the parameter level for linear policy $u = kz + b$. Each future attacked observation z induced a safe-action kernel over ambiguity interval $[z - \rho, z + \rho]$. Requiring $kz + b$ to lie in every kernel produced linear halfspace constraints on (k, b). The parameter gate projected poisoned candidates onto this certified set.

Fig. 9 shows the parameter-level frontier. At $\rho = 0.7$, current-action filtering had stale-policy rate 1.0. The parameter gate kept the rate at zero and permitted two of five updates. It permitted one constrained update at $\rho = 0.8$ and froze at $\rho = 0.85$. Fig. 10 repeats the comparison for the matrix parameterization.

We applied the same construction to policy $u = k_1 z_1 + k_2 z_2 + b$. Each grid observation induced a plausible-state box and a one-step safe-action interval. These intervals produced halfspaces in (k_1, k_2, b). At $\rho = 0.5$, current-action filtering had stale-policy rate 1.0. The parameter gate kept the rate at zero and admitted all five projected updates.

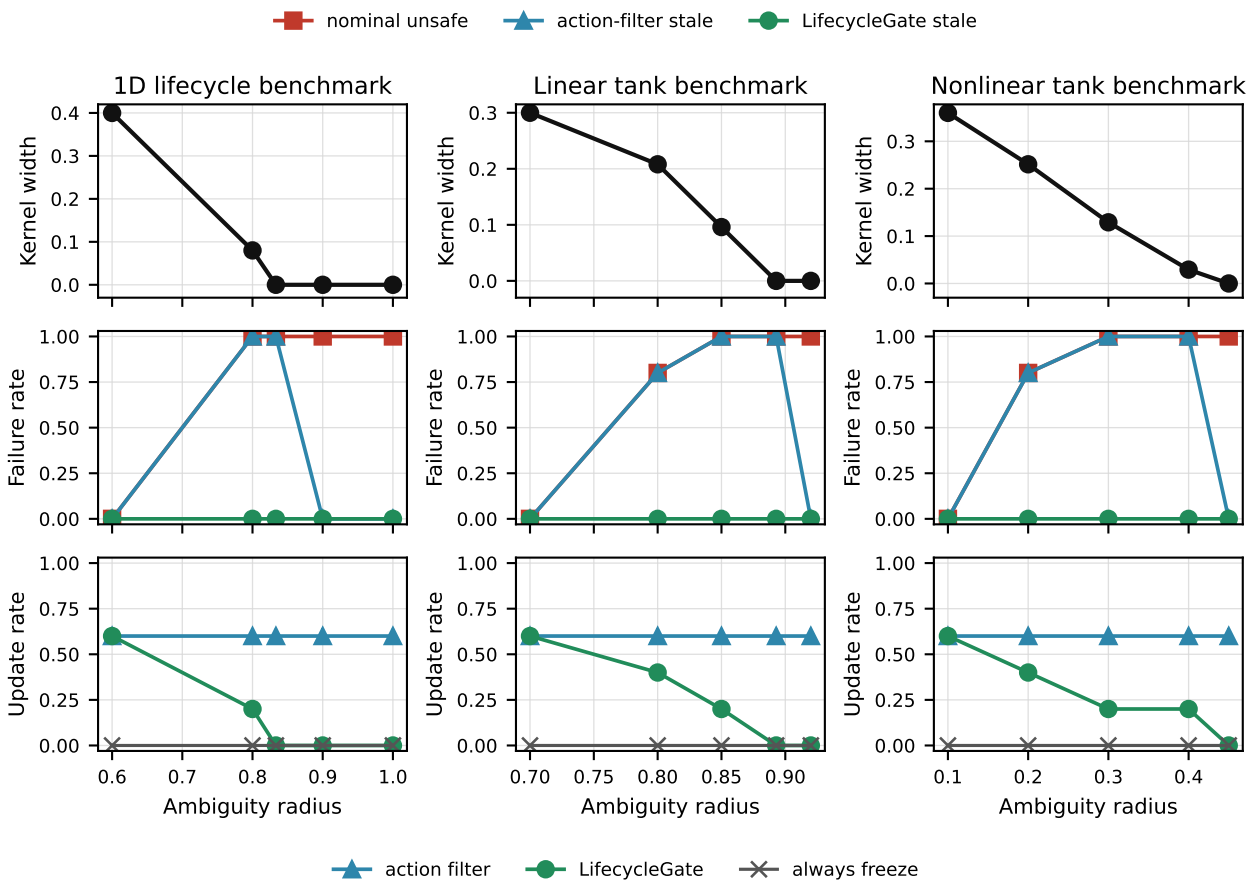

Figure 8: Lifecycle-gate benchmark results. Increasing ambiguity shrinks the certified kernels of the one-dimensional, linear-tank, and nonlinear-tank models. LifecycleGate constrains or freezes updates when action-only filters would leave the updated policy outside the kernel.

G.6 Adaptive Attacker Stress Test

We next allow a finite-search white-box attacker to select among poisoned update directions. Its objectives maximize stale-policy, unsafe-action, or forced-freeze outcomes. For forced freeze, the attacker also selects the budget-feasible ambiguity radius that most reduces the certified set.

Fig. 11 preserves the lifecycle separation under adaptive direction search. At high ambiguity, the robust action filter protected the applied action but left the raw policy stale. LifecycleGate kept unsafe, stale, and uncertified-learning rates at zero while allowing feasible projected updates. At $\rho = 0.84$, the forced-freeze objective made the common kernel empty. The robust filter recorded five uncertified learner updates, whereas LifecycleGate performed none.

G.7 Safe-Control-Gym Cartpole Semantics

We next instantiate the lifecycle condition in the Safe-Control-Gym cartpole benchmark. The official LQR is augmented with a two-parameter linear residual learner. The attacker injects continuous observation FDI and optimizes a cross-entropy-method (CEM) objective over stale-policy, unsafe-action, and forced-freeze margins. The attacked observation z and radius ρ induce the plausible box $X_{\mathcal{A}}(z) = \{x : \|x - z\|_\infty \leq \rho\}$. We sample this box and evaluate the official CasADi cartpole dynamics over an action grid to compute $K_\Gamma(z)$. A reference run uses the official cartpole CBF and its sampled Lie-derivative condition.

This experiment is a direct semantic implementation of Theorem 1 and Proposition 2. The future-history abstraction $\mathcal{Z}$ is a local grid of future attacked observations around the current attacked observation. For each $z' \in \mathcal{Z}$, the robust kernel produces an interval $K_\Gamma(z') = [\ell_{z'}, u_{z'}]$. The LQR term is fixed, while the residual learner is linear in $(\theta_{\text{gain}}, \theta_{\text{bias}})$, so requiring the updated residual policy to stay inside all future kernels yields the halfspace constraints used by LifecycleGate.

Fig. 12 separates three effects. Action filtering removed current constraint violations without certifying the updated residual policy. Always-freeze avoided stale certification but admitted no learning. LifecycleGate projected feasible poisoned updates into the future-history halfspace set, preserving update availability before the freeze frontier. At larger ambi-

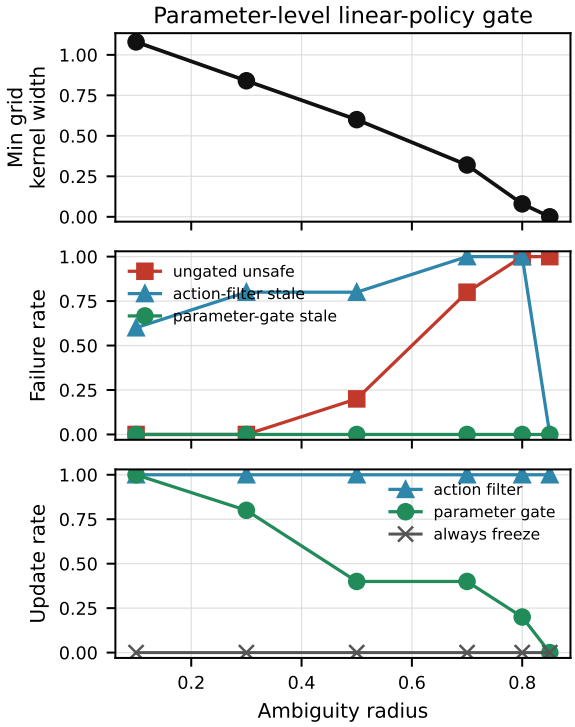

Figure 9: Parameter-level gate for linear policy $u = kz + b$. Current-action filtering leaves the future action map stale, whereas projection onto the certified halfspace set constrains the full update.

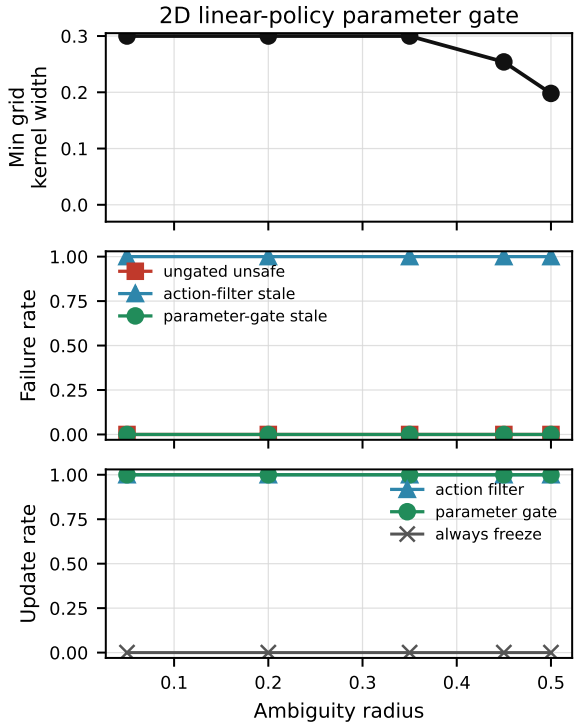

Figure 10: Parameter-level gate for the two-dimensional linear tank policy. The certified set intersects future-history halfspaces, and the gate projects the poisoned update into that set.

guity radii, the kernel became empty and LifecycleGate froze the update. These results instantiate the certificate semantics in a standard control benchmark; the evaluation does not represent an industrial deployment.

H Certificate Coverage and Model Mismatch

Reporting zero violations only on certificate-admitted states can be vacuous. We therefore lock a separate CasADi–PyBullet confusion audit before execution. It reconstructs the 12 clean, poisoned, commit, and freeze snapshots from the original traces, draws 144 states from three disjoint source seeds, performs no certificate admission, and compares horizons 20/50/100 and guard margins 0/0.001/0.003/0.005/0.01. The primary configuration is the pre-existing 100-step, 0.003-guard commit configuration; the grid is characterization rather than a selector.

Table 9 evaluates 1728 snapshot–state pairs. CasADi certifies 1293; none violates the PyBullet state box within 100 steps. Clean, commit, and freeze coverage is 0.986, 0.944, and 1.000. The raw poisoned snapshots physically violate 403/432 pairs and have certificate coverage 0.062, so their low coverage is informative rejection, not vacuous evidence for a safe deployed snapshot. Across all mechanisms the certificate conservatively rejects five physically safe pairs. Maximum action-interface error and actuator saturation are zero.

The grid exposes a model-mismatch boundary. Without a guard, poisoned snapshots produce 1, 2, and 1 false acceptances at horizons 20, 50, and 100. Every locked positive guard removes observed false acceptance, with increasing false rejection as the guard grows. The value 0.003 was the declared primary configuration before this audit. Zero observed false acceptance on the finite evaluation distribution does not establish continuous-state invariance.

I Unified Paired Security–Availability Frontier

J Closest-Work Comparison

Table 9: CasADi–PyBullet coverage at the pre-existing 100-step, 0.003-guard configuration. Each row pools three learner snapshots over the same 144 states.

Snapshot	Pairs	Physical viol.	Certified	False accept	False reject	Coverage
Clean REINFORCE	432	6	426	0	0	0.986
Poisoned action-only	432	403	27	0	2	0.062
Poisoned commit	432	21	408	0	3	0.944
Always-freeze	432	0	432	0	0	1.000

Table 10: Unified paired security–availability frontier. Wilson 95% intervals are $[0, 0.138]$ for every 0/24 violation count and $[0.798, 0.993]$ for poisoned raw. Intervention is a fraction of executed steps. † denotes reward truncated at the first safety failure and therefore not a utility comparison.

Mechanism	Viol./rollouts	Complete	Mean reward	Interv.	Latency med./p95 (ms)
Clean REINFORCE	0/24	24/24	−0.024	0.0%	0.027/0.029
Poisoned raw	23/24	1/24	−0.189†	0.0%	0.027/0.029
Poisoned commit	0/24	24/24	−0.056	0.0%	0.026/0.029
Always-freeze	0/24	24/24	−0.023	0.0%	0.026/0.028
Permanent shield	0/24	24/24	−0.195	21.3%	221.925/238.216
Official MPSC	0/24	24/24	−0.457	100.0%	59.657/76.055

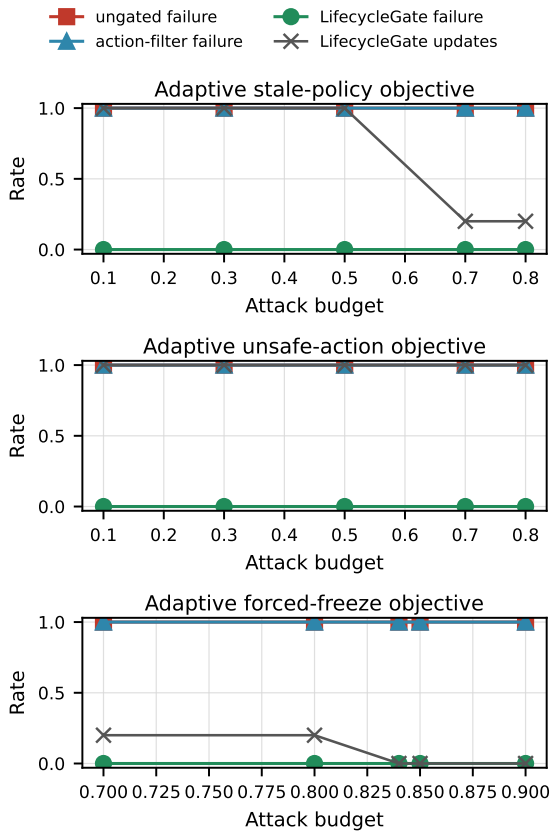

Figure 11: Finite-search attacker against the scalar parameter gate. The attacker selects update directions or ambiguity radii for three objectives. LifecycleGate freezes when the certified kernel is empty.

Table 11: Closest work along the three axes combined in this paper: adversarial online update data, removal of runtime-assurance authority, and independent evidence for the released raw policy.

Work	Adversarial update log	Authority removed	Independent raw-policy evidence
Simplex / Neural Simplex [1, 8]	no	no (monitor resident)	n/a (composition certified)
Bb-Simplex [9]	no (trusted retraining)	no	n/a
Policy repair / MPSC [3, 10]	no (trusted evidence)	repaired, verified	repaired
Hsu et al. [5]	no	partial	yes (transfer evidence)
Carr et al. [4]	no	yes (sudden/smooth)	empirical retirement
BADControl [38]	operational data, physical trigger	n/a	n/a
Jiang et al. [37]	yes (backdoor, triggered)	no (shield resident)	n/a
Controlled study	**yes (reward record)**	**yes (raw release)**	**transition evidence tested**
Third-party case study	**yes (reward record)**	**yes (Carr retirement)**	**transition evidence tested**

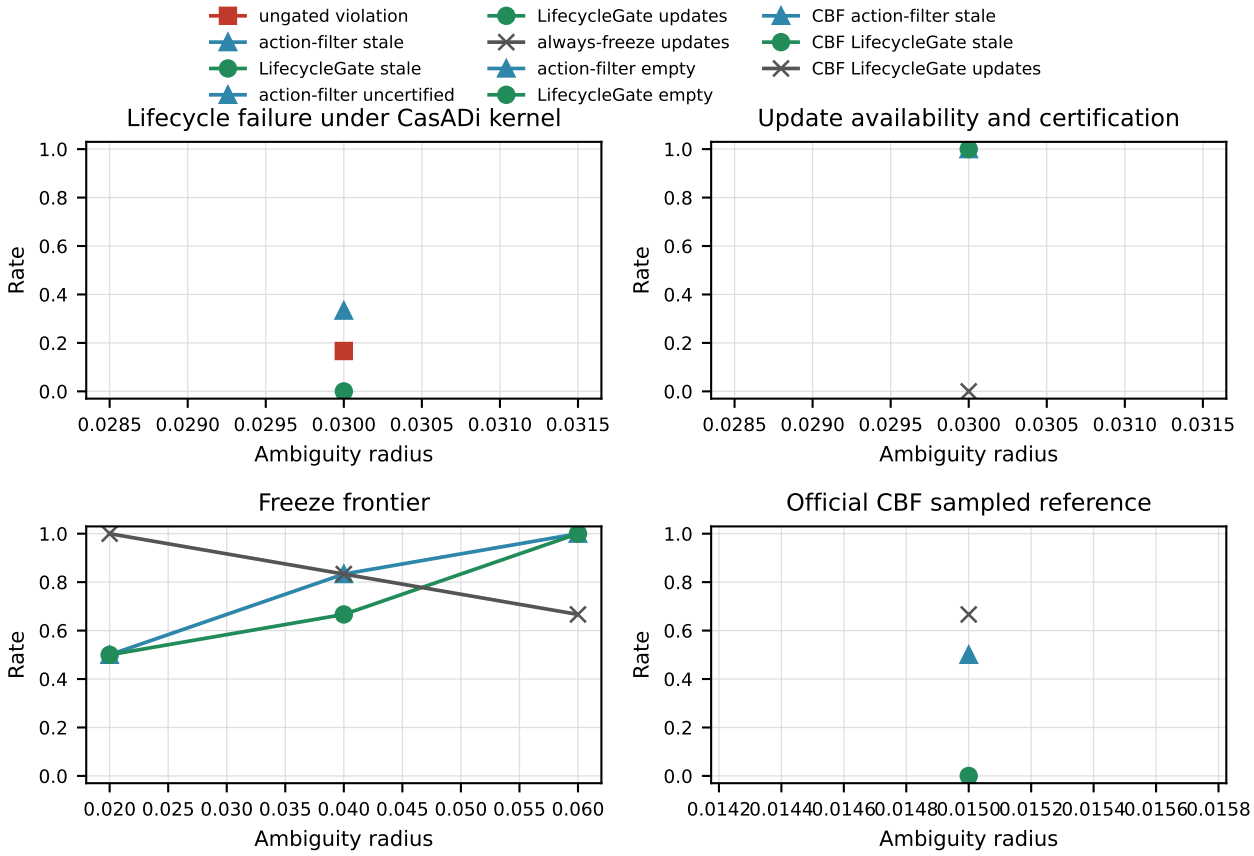

Figure 12: Safe-Control-Gym cartpole lifecycle evidence from a two-run sweep. The panels compare the CasADi robust kernel, an Euler attack-search surrogate, and a sampled CBF reference. At $\rho = 0.03$, LifecycleGate eliminated stale and uncertified updates while retaining update availability. The kernel became empty as ambiguity increased.

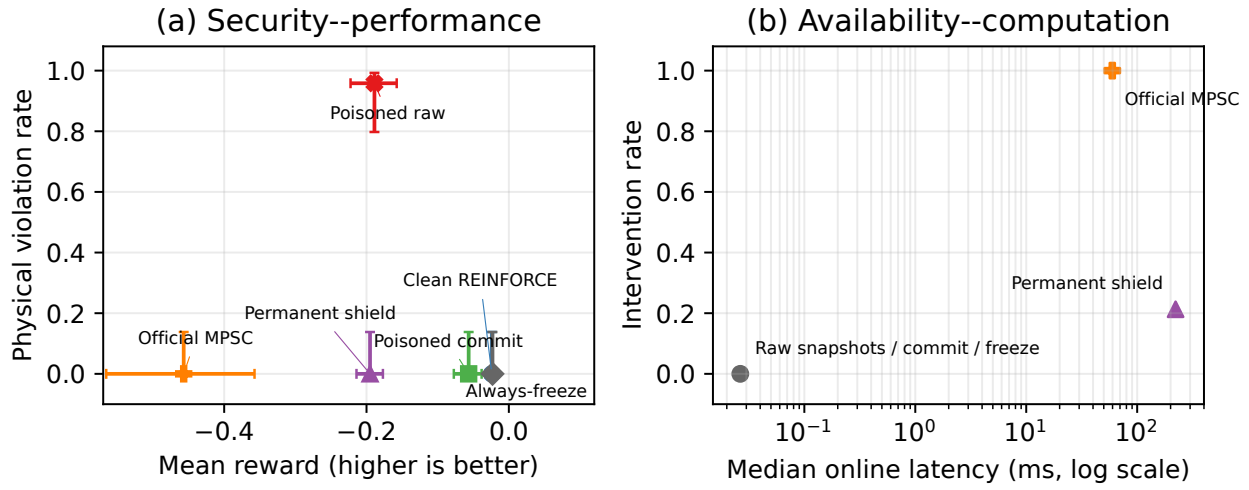

Figure 13: The unified frontier on the same paired state blocks. Error bars in (a) are 95% Wilson intervals for violation rate and paired-block bootstrap intervals for mean rollout reward, conditional on the three locked learner snapshots. Panel (b) separates the negligible raw policy latency from the permanently-online protection cost; raw, commit, and freeze mechanisms overlap near 0.027 ms and zero intervention.